

目录

- 1 实验目的**
- 2 实验平台及环境**
- 3 实验内容**
- 4 实验步骤与结果分析**
 - 4.1 前置步骤
 - 4.2 创建和管理索引
 - 4.2.1 创建普通索引
 - 4.2.2 管理索引
 - 4.2.3 索引合理性分析
 - 4.3 创建 4 类索引
 - 4.3.1 创建唯一索引
 - 4.3.1.1 步骤
 - 4.3.1.2 性能对比分析
 - 4.3.2 创建多字段索引
 - 4.3.2.1 步骤
 - 4.3.2.2 性能对比分析
 - 4.3.3 创建部分索引
 - 4.3.3.1 优化器异常选择分析
 - 4.3.3.2 步骤
 - 4.3.3.3 性能对比分析
 - 4.3.4 创建表达式索引
 - 4.3.4.1 性能对比分析
 - 4.4 创建和管理视图
 - 4.4.1 创建视图
 - 4.4.2 管理视图
- 5 实验总结与心得体会**

实验五 索引与视图

1 实验目的

- 1. 通过实验让学生熟悉并了解 GaussDB 数据库的基本机制与操作；
- 2. 通过索引管理、视图管理等管理的操作，让学生熟悉并了解 DAS 环境下如何使用 GaussDB；

2 实验平台及环境

本实验环境为华为云 GaussDB 数据库，具体的配置如表 2-1 所示：

表 2-1 实验环境

配置项	参数
华为云	软件版本：GaussDB v2.0-8.210.0
	设备型号：GaussDB 8 核 64 G
数据库	PostgreSQL

3 实验内容

- 1. 本实验通过索引管理等操作，让学生熟悉并了解 DAS 环境下如何使用 GaussDB 管理索引；
- 2. 本实验通过视图管理等操作，让学生熟悉并了解 DAS 环境下如何使用 GaussDB 管理视图；

4 实验步骤与结果分析

为保证报告叙述的连续性，我将“实验步骤”与“实验结果分析”合并为一节，在每个操作步骤之后给出对应的结果和对实验结果的分析。

4.1 前置步骤

1. 登录华为云的 IAM 账户；
2. 登录 GaussDB 数据库实例；
 - (a) 在控制台选择并进入“云数据库 GaussDB”；
 - (b) 选择“gauss-6025-11”实例并登录；

4.2 创建和管理索引

要求：

1. 索引可以提高数据的访问速度，但同时也增加了插入、更新和删除表的处理时间；所以是否要为表增加索引，索引建立在哪些字段上，是创建索引前必须要考虑的问题；需要分析应用程序的业务处理、数据使用、经常被用作查询条件或者被要求排序的字段来确定是否建立索引；
2. GaussDB 支持 4 种创建索引的方式：唯一索引、多字段索引、部分索引、表达式索引；

对应步骤：

4.2.1 创建普通索引

1. 在“订单明细基本信息表”给 `order_id` 添加**普通索引**，对应的 SQL 语句见下文，执行结果如图 4-1 所示；
2. 在“商品基本信息表”中，为 `product_name` 添加**唯一索引**，对应的 SQL 语句见下文，执行结果如图 4-2 所示；
3. 在“订单基本信息表”中，为 `order_id` 和 `create_time` 添加**复合索引**，对应的 SQL 语句见下文，执行结果如图 4-3 所示；
4. 在“用户基本信息表”中，对年龄在 20 岁以上的用户添加**部分索引**，对应的 SQL 语句见下文，执行结果如图 4-4 所示；

上文涉及的 SQL 语句如下：

```

1  -- 在“订单明细基本信息表”给 order_id 添加普通索引
2  CREATE INDEX idx_order_item_order_id ON "订单明细基本信息表" ("order_id");
3
4  -- 在“商品基本信息表”中，为 product_name 添加唯一索引
5  CREATE UNIQUE INDEX uk_product_name ON "商品基本信息表" ("name");
6
7  -- 在“订单基本信息表”中，为 order_id 和 create_time 添加复合索引
8  CREATE INDEX idx_order_id_create_time ON "订单基本信息表" ("order_id",
9  "create_time");
10
11 -- 在“用户基本信息表”中，对年龄在 20 岁以上的用户添加部分索引
12 CREATE INDEX idx_user_age_over_20 ON "用户基本信息表" ("age") WHERE age > 20;

```

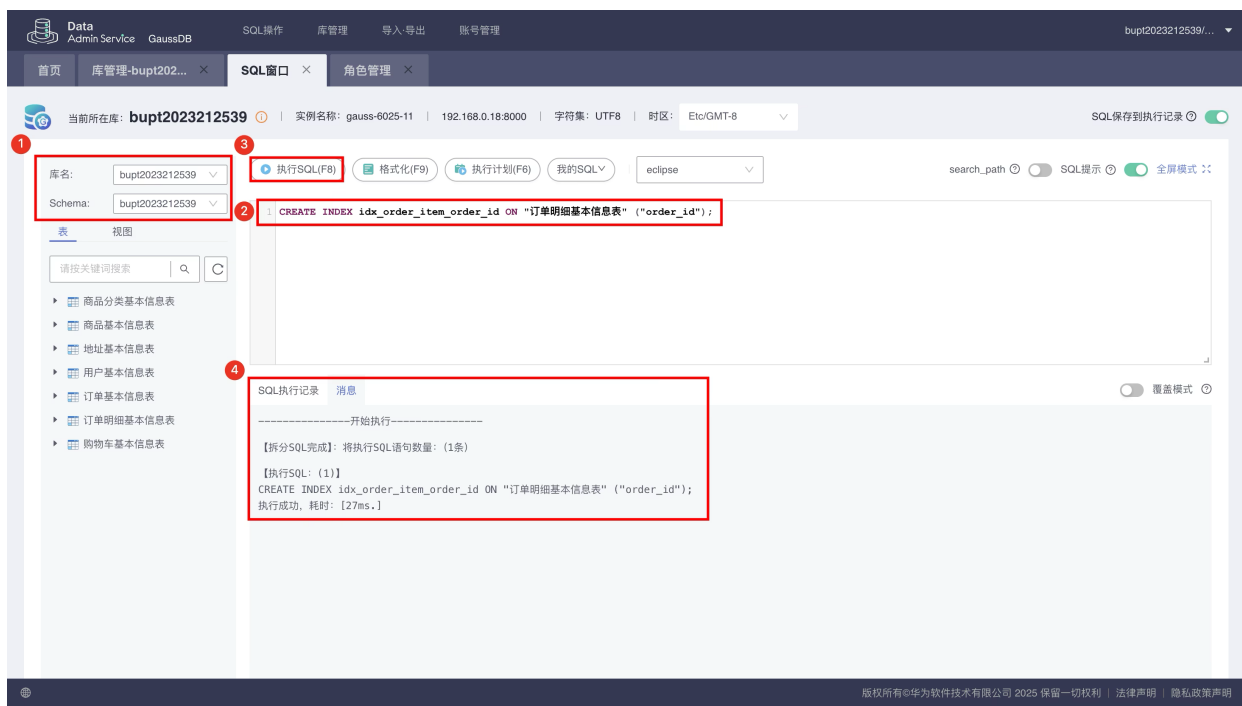


图 4-1 在“订单明细基本信息表”给 order_id 添加普通索引

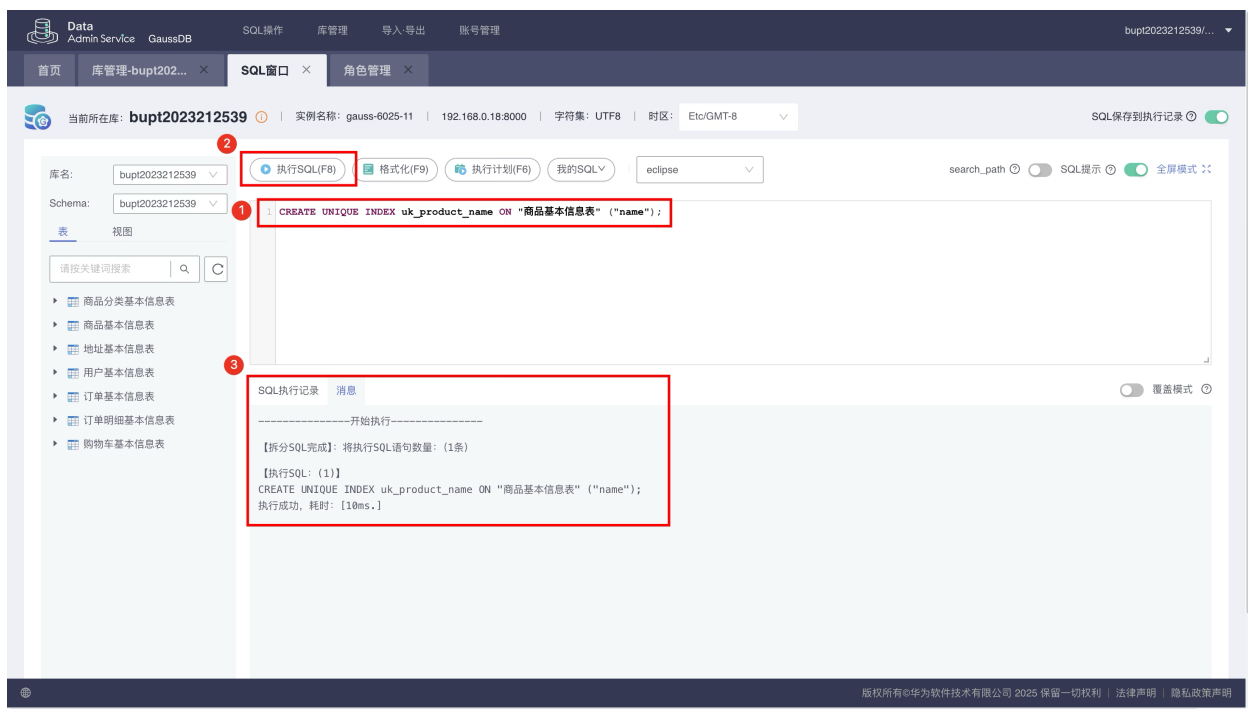


图 4-2 在“商品基本信息表”中，为 product_name 添加唯一索引

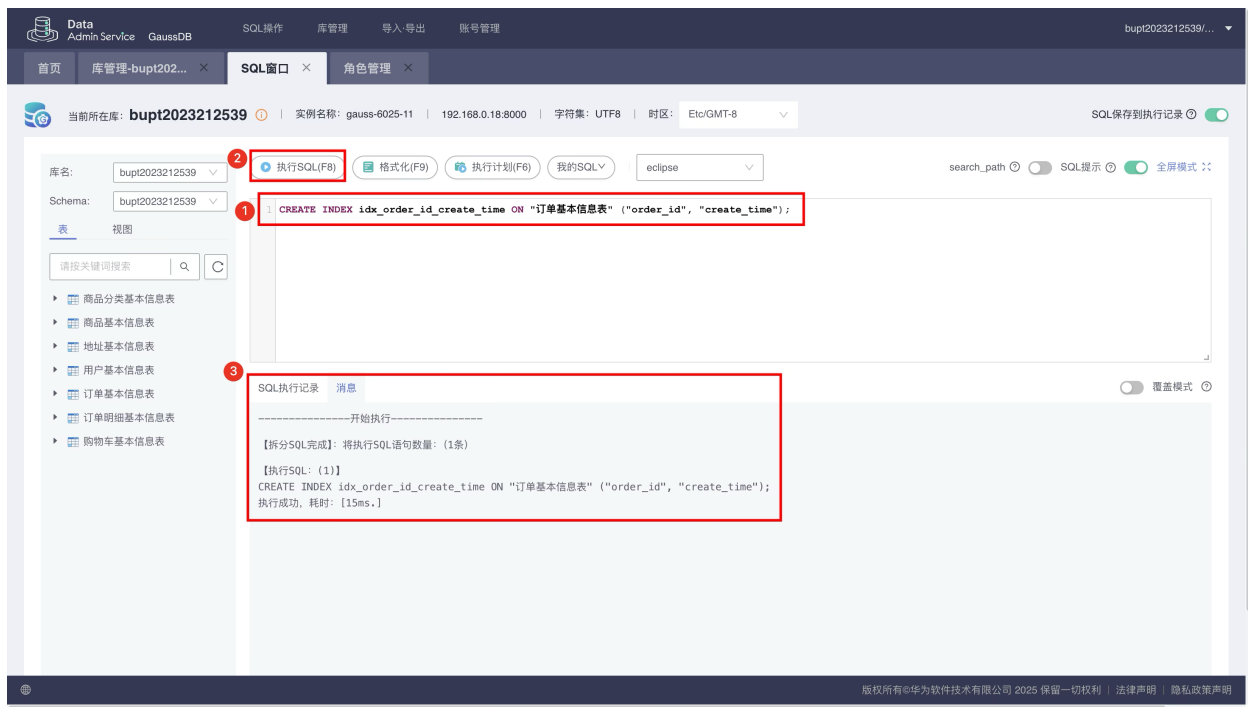


图 4-3 在“订单基本信息表”中，为 order_id 和 create_time 添加复合索引

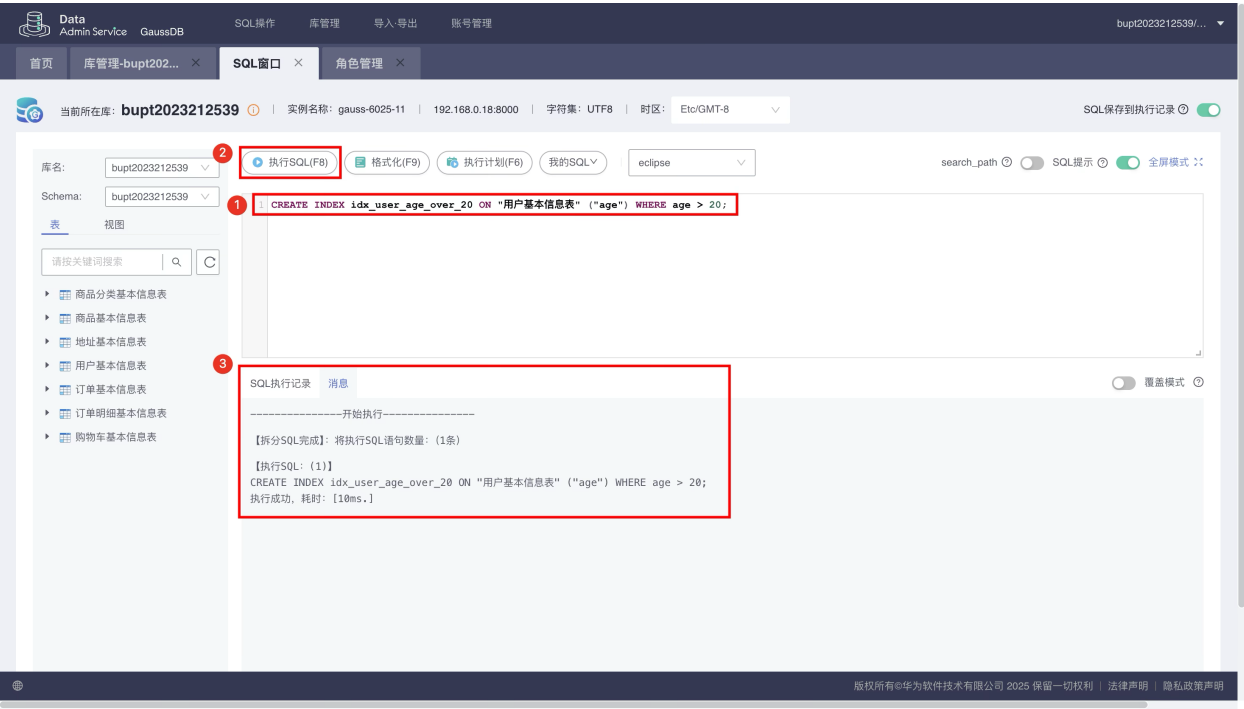


图 4-4 在“用户基本信息表”中，对年龄在 20 岁以上的用户添加部分索引

4.2.2 管理索引

1. 查询索引：创建索引后刷新页面，左下角会显示表视图，单击 indexes 显示当前表的所有索引，如图 4-5 所示；

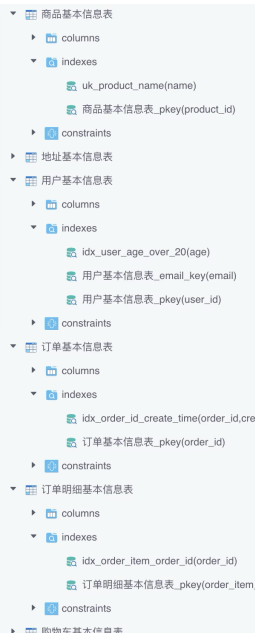


图 4-5 索引创建结果查看

2. 删除索引：删除“订单明细基本信息表”的 `order_id` 普通索引 `idx_order_item_order_id`，对应的 SQL 语句见下文，执行结果如图 4-6 所示，由左侧结果可见，索引 `idx_order_item_order_id` 被正确删除；

上文涉及的 SQL 语句如下：

```
1 -- 删除“订单明细基本信息表”的 order_id 普通索引 idx_order_item_order_id
2 DROP INDEX idx_order_item_order_id;
```

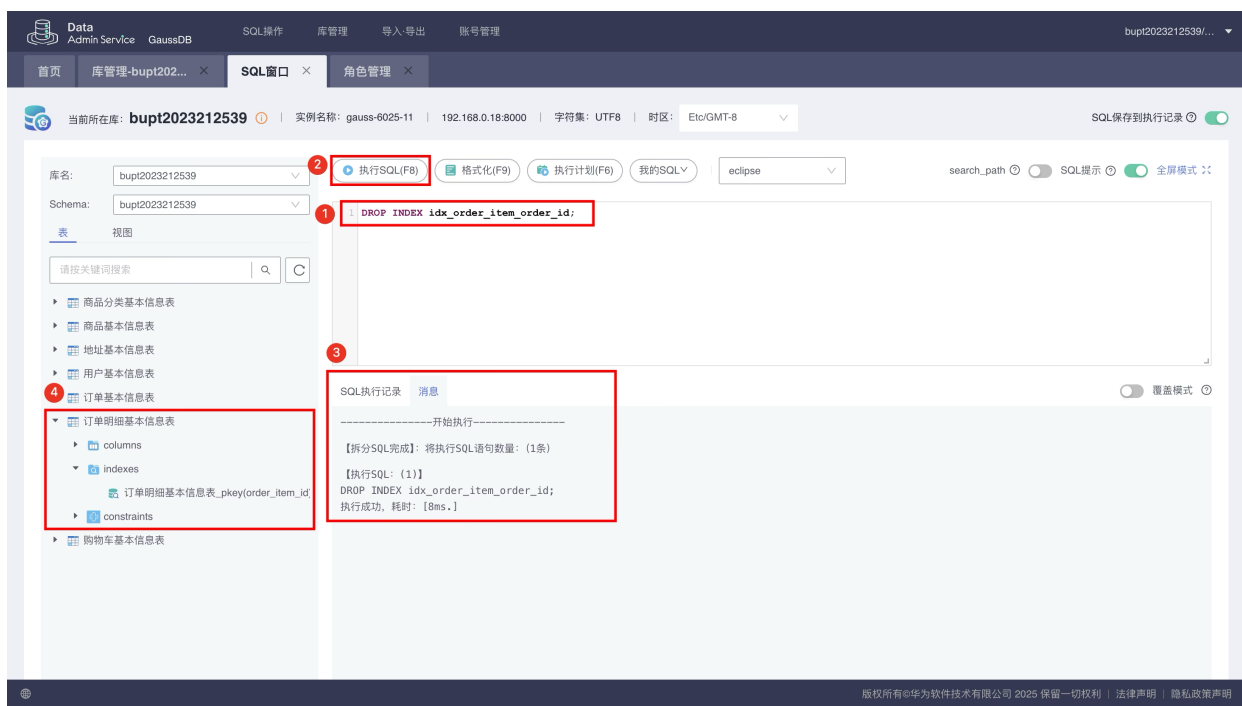


图 4-6 删除“订单明细基本信息表”的普通索引

4.2.3 索引合理性分析

对于上面创建的索引，分析如下：

1. 在“订单明细基本信息表”给 `order_id` 添加普通索引

“订单明细基本信息表”记录了每个订单（由 `order_id` 标识）包含的具体商品。业务中，查询一个特定订单的所有明细是极其常见的操作，例如，用户查看订单详情。

其中，`order_id` 是该表的外键，它频繁被用作连接条件（例如与“订单基本信息表”连接）或查询条件（`WHERE order_id = X`）。

因此，在作为外键且频繁用于查询和连接的字段上建立普通索引，可以显著加速根据订单 ID 查找其所有明细的速度，该普通索引创建合理。

2. 在“商品基本信息表”中，为 `product_name` 添加唯一索引

唯一索引（Unique Index）的主要作用是确保数据的唯一性和加速基于该字段的查询。

如果电商平台要求商品名称是唯一的，那么添加唯一索引是合理的，它可以作为数据库层面的数据约束。

同时，如果业务中频繁需要根据商品名称进行精确查询（例如，`WHERE name = 'Dell XPS 13 Laptop'`），唯一索引也能提供比普通索引更快的查找速度，因为它知道最多只有一个结果。

因此，如果业务要求商品名称必须唯一，且经常通过名称查找商品，则创建唯一索引是合理的。但是，如果商品名称允许重复，则应使用普通索引或不建索引。

3. 在“订单基本信息表”中，为 `order_id` 和 `create_time` 添加复合索引

其中，`order_id` 是该表的主键（`PRIMARY KEY`），而主键本身已经自动具有唯一的、聚集的索引。

复合索引应建立在那些经常一起出现在 `WHERE` 或 `ORDER BY` 子句中的字段上，且通常不包含主键。

由于 `order_id` 已经是主键，这个复合索引并不合理。它不会带来性能提升，反而会增加数据写入时的开销。

如果需要优化查询，应考虑为单独的 `create_time` 字段建立索引，用于加速基于时间范围的查询，例如，查询某时间段内的所有订单。

对应的 SQL 语句如下：

```
1 CREATE INDEX idx_order_create_time ON "订单基本信息表" ("create_time");
```

4. 在“用户基本信息表”中，对年龄在 20 岁以上的用户添加部分索引

部分索引（Partial Index）只对表中满足特定条件的数据行建立索引。

假设业务逻辑高度关注 20 岁及以上用户，导致涉及 `age > 20` 的查询非常频繁。则通过部分索引，可以极大地减小索引的体积（只索引一部分数据），从而提高查询速度，并减少插入、更新 20 岁以下用户时的索引维护开销。

因此，如果业务确实集中在特定年龄段，这个部分索引合理且高效。

4.3 创建 4 类索引

要求：

- 对“订单明细基本信息表”和“订单基本信息表”创建“唯一索引/多字段索引/部分索引/表达式索引”；
- 并尝试比较未建索引与创建索引后，查询效率的不同；

对应步骤：

4.3.1 创建唯一索引

4.3.1.1 步骤

1. 在“订单明细基本信息表”中，确保每个订单中同一商品只能出现一次，即 `order_id` 和 `product_id` 的组合唯一。在“订单明细基本信息表”中，给 `order_id` 和 `product_id` 添加唯一索引，对应的 SQL 语句见下文，执行结果如图 4-9 所示；

注：这里直接执行创建唯一索引操作会报如图 4-7 所示的错误“**ERROR: Failed to create a unique index "idx_order_item_unique". Detail: Key (order_id, product_id)=(4965, 20) is duplicated.**”，由报错信息可见，表中存在重复组合，因此需要先对重复组合进行删除，对应的 SQL 语句见下文，执行结果如图 4-8 所示。

上文涉及的 SQL 语句如下：

```
1  -- 删除重复组合，只保留最早一条
2  DELETE FROM "订单明细基本信息表" a
3  USING "订单明细基本信息表" b
4  WHERE a.ctid > b.ctid
5         AND a.order_id = b.order_id
6         AND a.product_id = b.product_id;
7
8  -- 创建唯一索引
9  CREATE UNIQUE INDEX idx_order_item_unique ON "订单明细基本信息表" ("order_id",
"product_id");
```

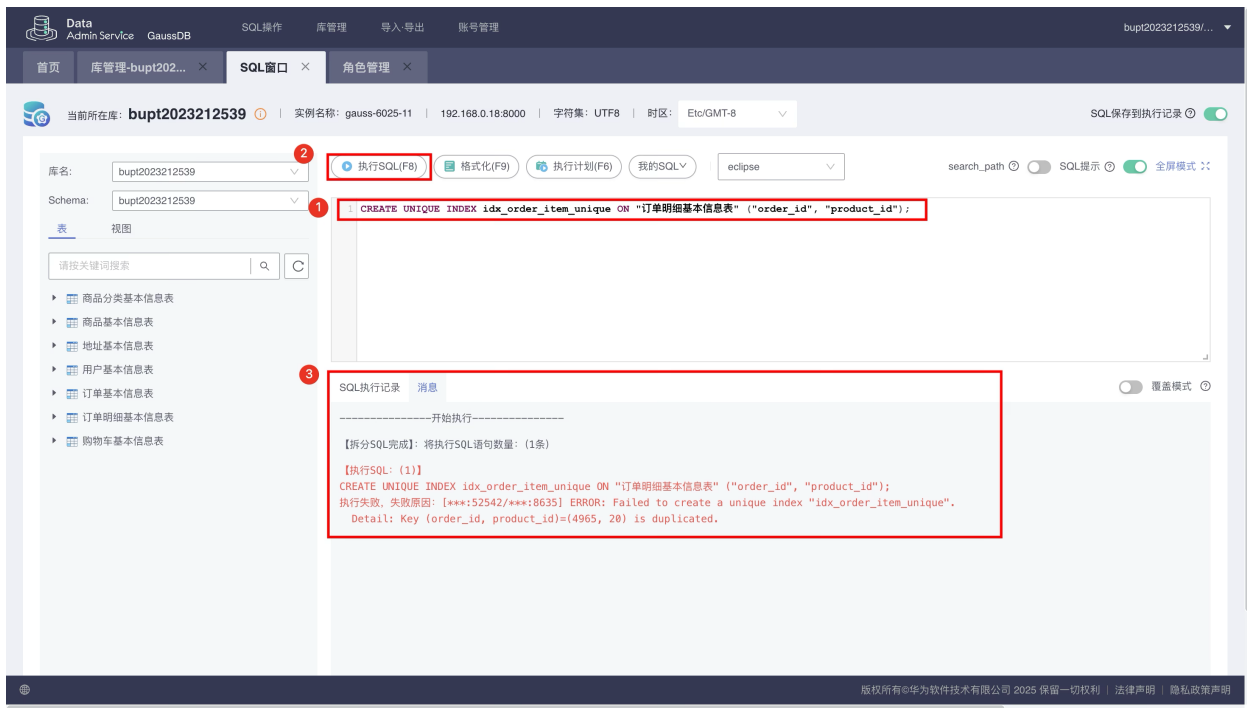


图 4-7 未删除重复组合前创建唯一索引报错

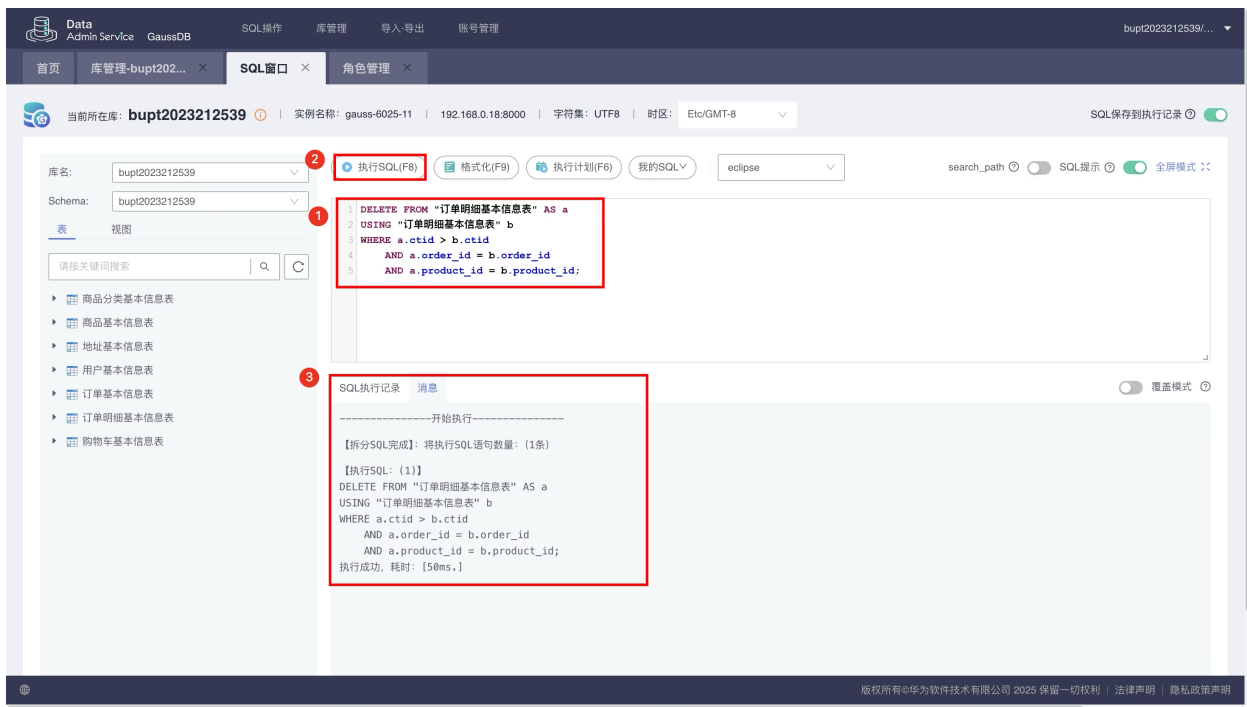


图 4-8 删除“订单明细基本信息表”中的重复组合

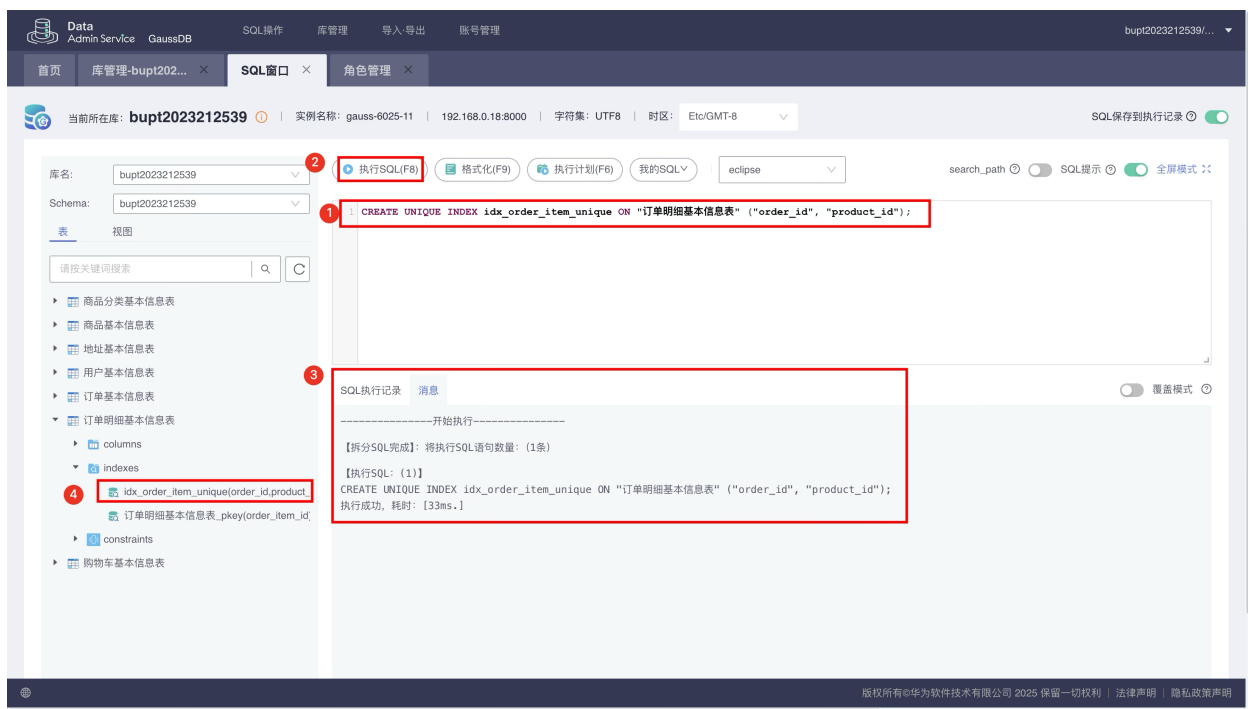


图 4-9 在“订单明细基本信息表”中，给 order_id 和 product_id 添加唯一索引

2. 比较未建唯一索引与创建唯一索引后，查询效率的不同。这里，我使用 `EXPLAIN ANALYZE` 命令来执行测试查询，它会显示查询执行计划和实际运行时间（Total runtime），并选择了一条真实信息（`order_id = 403,` `product_id = 27`）作为查询对象。

- (a) 测量无索引时的查询性能（基准测试），对应的 SQL 语句见下文，执行结果如图 4-10 所示；
- (b) 创建唯一索引，见上文；
- (c) 测量有索引时的查询性能，在唯一索引创建完成后，再次执行完全相同的查询，并记录新的运行时间进行比较，执行结果如图 4-11 所示；

上文涉及的 SQL 语句如下：

```
1  -- 在添加唯一约束/索引之前和之后分别执行
2  EXPLAIN ANALYZE
3  SELECT quantity, price
4  FROM "订单明细基本信息表"
5  WHERE order_id = 403 AND product_id = 27;
```

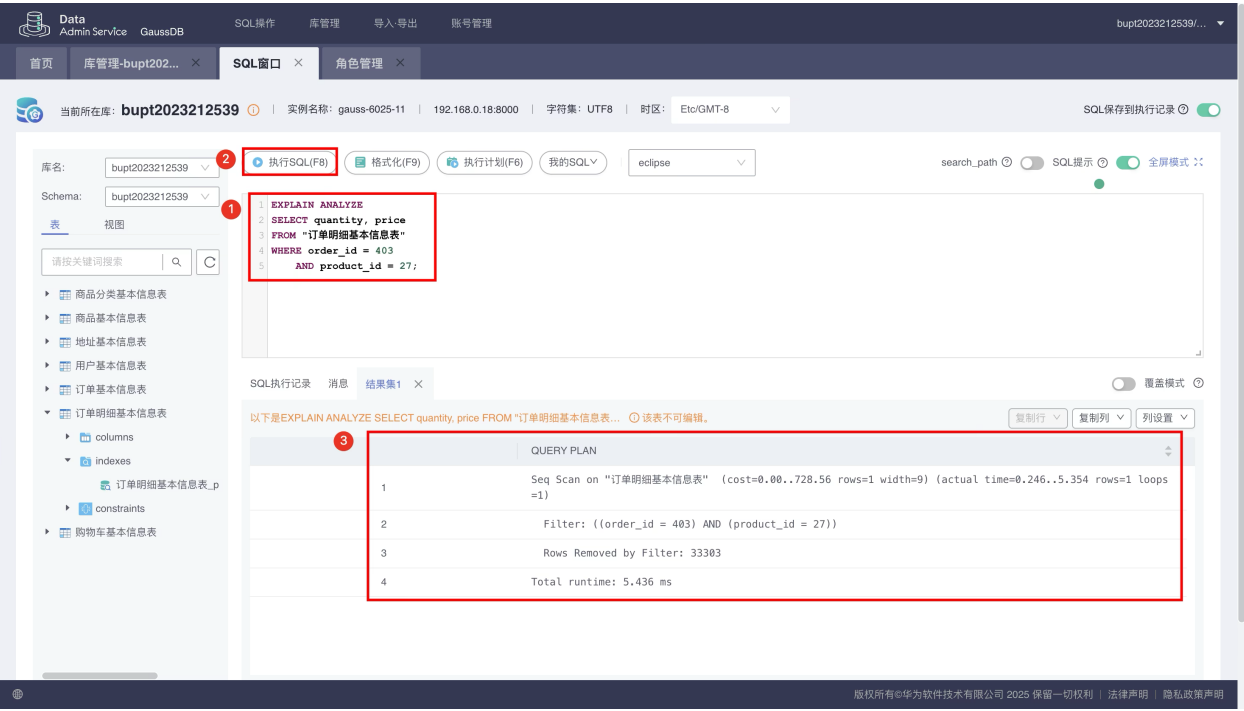


图 4-10 添加唯一索引前查询时间

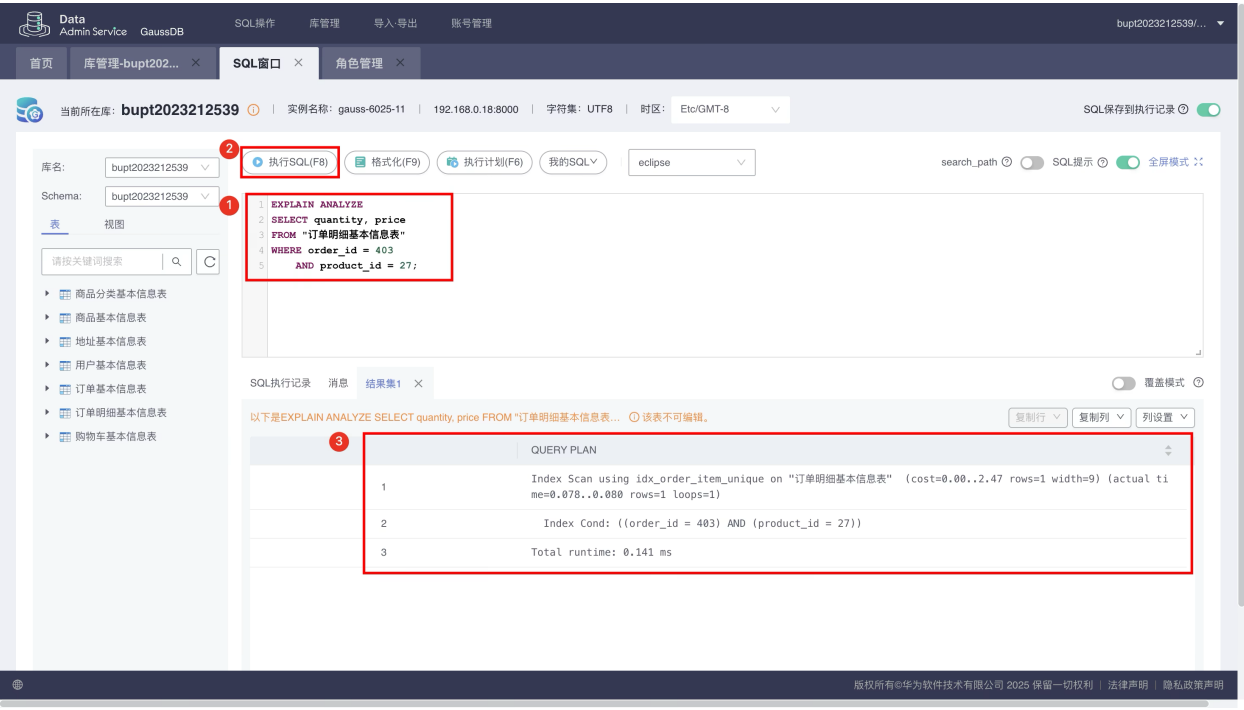


图 4-11 添加唯一索引后查询时间

4.3.1.2 性能对比分析

整理上图所示的结果如表 4-1 所示：

表 4-1 添加唯一索引前后查询效率对比

性能指标	未添加唯一索引 (Seq Scan)	添加唯一索引 (Index Scan)	性能提升
查询方法	顺序扫描 (Seq Scan)	索引扫描 (Index Scan)	
查询成本	0.00..728.56	0.00..2.47	大幅降低
实际执行时间	5.436 ms	0.141 ms	约 38.5 倍
过滤行数	33,303 行	直接通过索引定位	

从执行结果的对比来看，为 `(order_id, product_id)` 组合添加唯一索引后，查询性能获得了显著的提升。

未添加唯一索引时：

数据库使用了**顺序扫描** (`Seq Scan`)，这意味着它必须从表的第一行开始，逐行检查直到找到所有匹配 `(order_id = 403 AND product_id = 27)` 的记录。因此，数据库扫描了整张表，过滤掉了 33,303 行，只找到了 1 行匹配结果。总运行时间为 5.436 毫秒。在高并发场景下，这个延迟是不可接受的。

添加唯一索引后：

数据库使用了**索引扫描** (`Index Scan`)，直接利用在 `(order_id, product_id)` 上的唯一索引进行查找。

索引树的结构允许数据库直接跳转到包含目标 `order_id` 和 `product_id` 的数据块，避免了对主表数据的全面扫描，对应的估算成本从 728.56 降至 2.47，显示优化器认为执行效率极高。总运行时间仅为 0.141 毫秒。

综上，在“订单明细基本信息表”上为 `(order_id, product_id)` 添加唯一索引，极大地优化了查询性能，针对基于 `order_id` 和 `product_id` 的点查询，性能提升了约 **38.5 倍** ($5.436\text{ ms}/0.141\text{ ms} \approx 38.5$ 倍)。

该实验结果充分验证了在外键组合上建立索引（特别是查询频率高的场景），对于加速查询、降低数据库 I/O 成本的关键作用。

4.3.2 创建多字段索引

4.3.2.1 步骤

考虑以下业务场景：需要经常查询“订单明细基本信息表”中包含商品编号为 3 的商品，且订单编号为 11 的订单。

1. 因此，在“订单明细基本信息表”中，在字段 `order_id` 和 `product_id` 上定义一个多字段索引，对应的 SQL 语句见下文，执行结果如图 4-12 所示；

上文涉及的 SQL 语句如下：

```
1 CREATE INDEX idx_order_item_composite ON "订单明细基本信息表" (order_id, product_id);
```

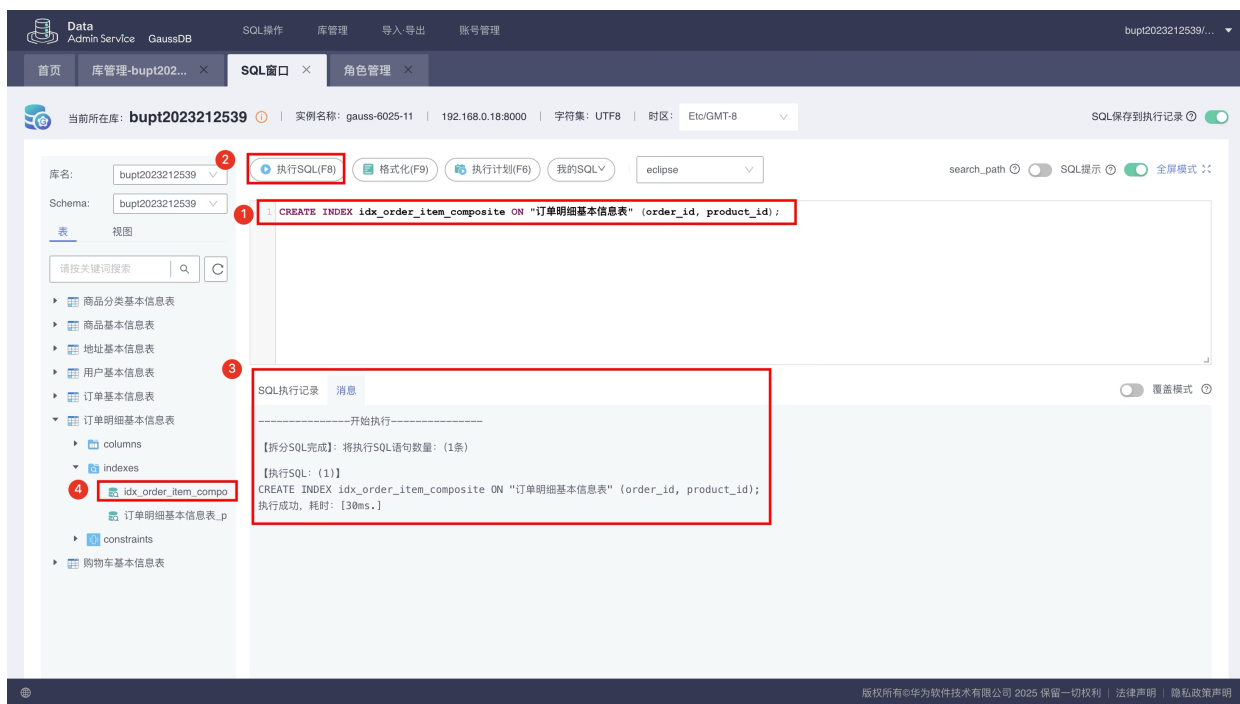


图 4-12 在“订单明细基本信息表”中，给 `order_id`, `product_id` 添加多字段索引

2. 比较未建多字段索引与创建多字段索引后，查询效率的不同。这里，我使用 `EXPLAIN ANALYZE` 命令来执行测试查询，它会显示查询执行计划和实际运行时间（Total runtime），并选择了一条真实信息（`order_id = 3,` `product_id = 11`）作为查询对象。
 - (a) 测量无索引时的查询性能（基准测试），对应的 SQL 语句见下文，执行结果如图 4-13 所示；
 - (b) 创建多字段索引，见上文；
 - (c) 测量有索引时的查询性能，在多字段索引创建完成后，再次执行完全相同的查询，并记录新的运行时间进行比较，执行结果如图 4-14 所示；

上文涉及的 SQL 语句如下：

```
1  -- 在添加唯一约束/索引之前和之后分别执行
2  EXPLAIN ANALYZE
3  SELECT *
4  FROM "订单明细基本信息表"
5  WHERE order_id = 11 AND product_id = 3;
```

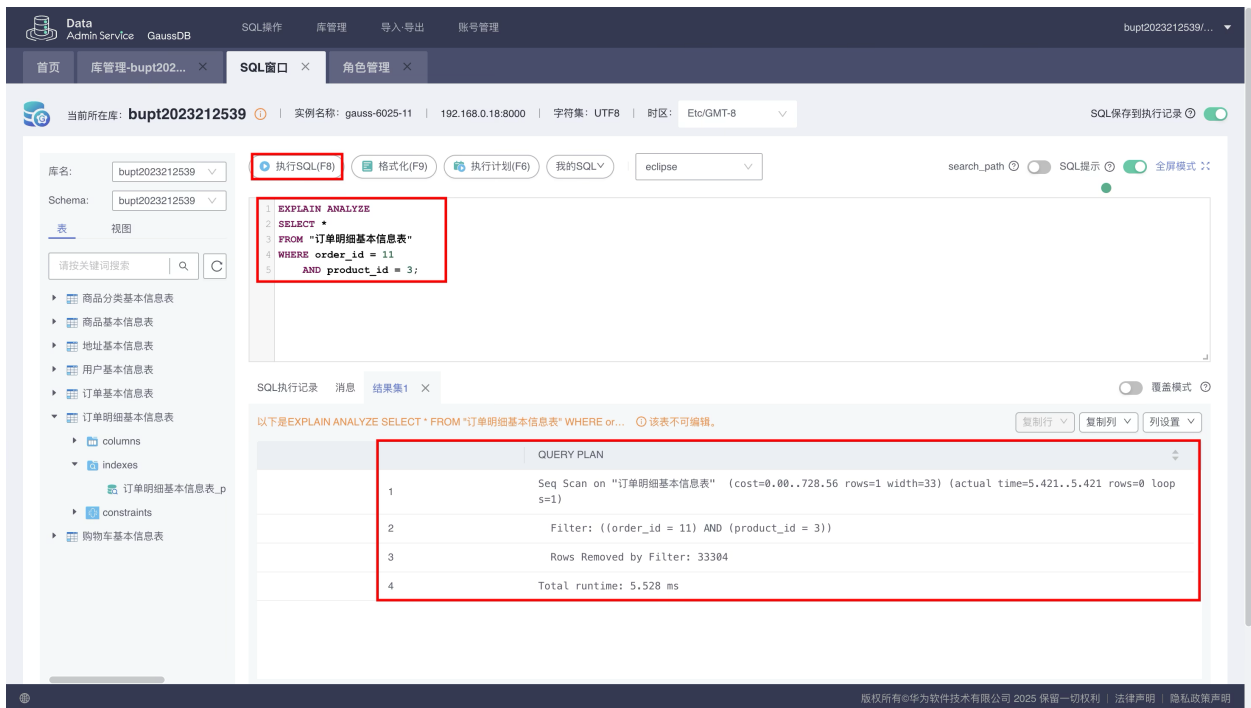


图 4-13 添加多字段索引前查询时间

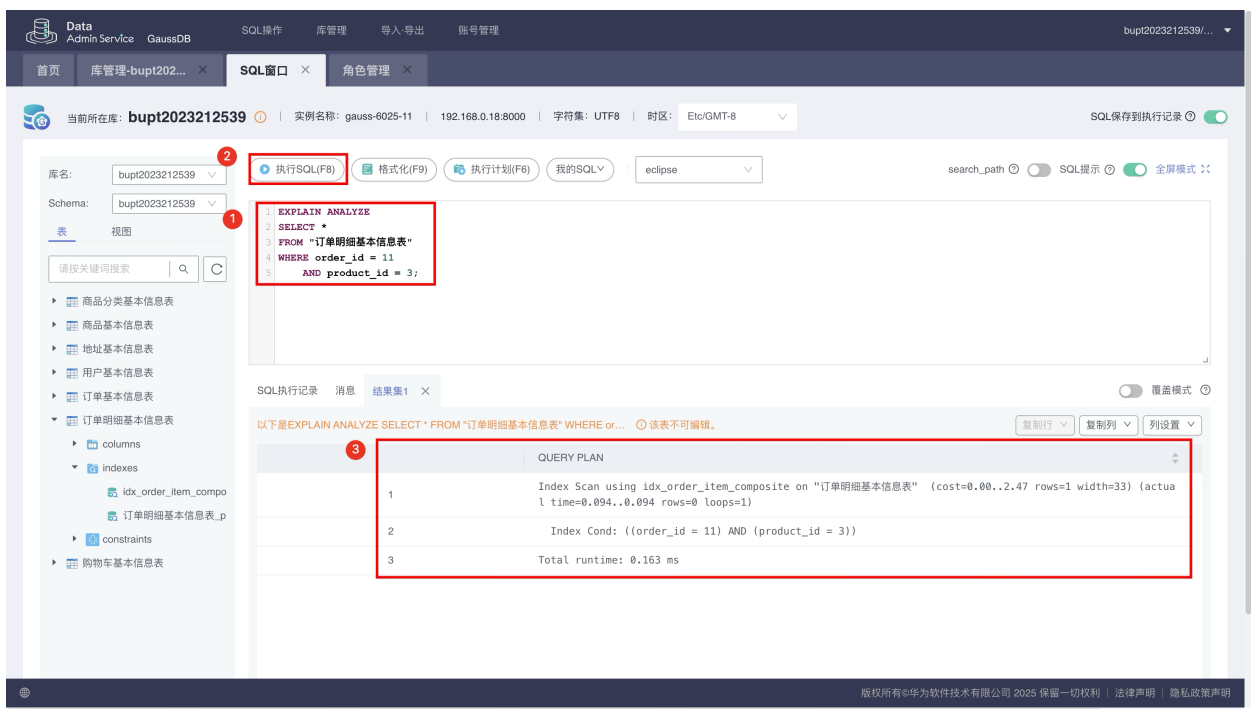


图 4-14 添加多字段索引后查询时间

4.3.2.2 性能对比分析

这次测试的查询对象是 `order_id = 11` 和 `product_id = 3`，但两次查询的 `rows=0` 结果表明该组合不存在。然而，即使是查询不存在的记录，索引也能极大地加速查找过程，整理上图所示的结果如表 4-2 所示：

表 4-2 添加多字段索引前后查询效率对比

性能指标	未建多字段索引 (Seq Scan)	已建多字段索引 (Index Scan)	性能提升
查询方法	顺序扫描 (Seq Scan)	索引扫描 (Index Scan)	
查询成本	0.00..728.56	0.00..2.47	大幅降低
实际执行时间	5.528 ms	0.163 ms	约 33.9 倍
扫描行数	33,304 行	直接通过索引定位	

实验结果验证了在查询条件 `WHERE order_id = X AND product_id = Y` 上建立多字段复合索引的巨大优势。

未建多字段索引时：

数据库选择了成本最高的顺序扫描 (`Seq Scan`)，数据库必须扫描表中的每一行（共 33,304 行），逐一检查 `order_id` 是否为 11 且 `product_id` 是否为 3。由于没有找到匹配的行 (`rows=0`)，它必须完整扫描整个表。总运行时间为 5.528 毫秒。

已建多字段索引后：

数据库识别并使用了新建的复合索引 `idx_order_item_composite`，执行索引扫描 (`Index Scan`)。索引树的结构允许数据库在极短的时间内确定：

- 该组合键在索引中是否存在。
- 因为找到了 0 行 (`rows=0`)，数据库只需在索引结构中快速导航并确认目标不存在，而不需要访问主表的其他数据。

估算成本从 728.56 降至 2.47。总运行时间降至 0.163 毫秒。

综上，即使是针对查询不存在的记录，多字段索引带来的性能提升依然是压倒性的：实际执行时间提升了约 **33.9 倍** ($5.528\text{ ms}/0.163\text{ ms} \approx 33.9$)。

索引将查询复杂度从扫描整个数据表（O(n) 复杂度，n 为行数）降低为在索引树中查找（通常是 O(logn) 复杂度），极大地加速了查询，充分证明了该多字段索引建立的合理性和高效性。

4.3.3 创建部分索引

4.3.3.1 优化器异常选择分析

这里，按照实验指导书要求，业务场景为需要经常查询总金额大于 1000 元的订单，但实际执行后发现，在创建了部分索引之后，查询仍然选择了顺序扫描 (Seq Scan)，如图 4-15 和图 4-16 所示。这表明优化器顺序扫描比使用创建的部分索引更高效。

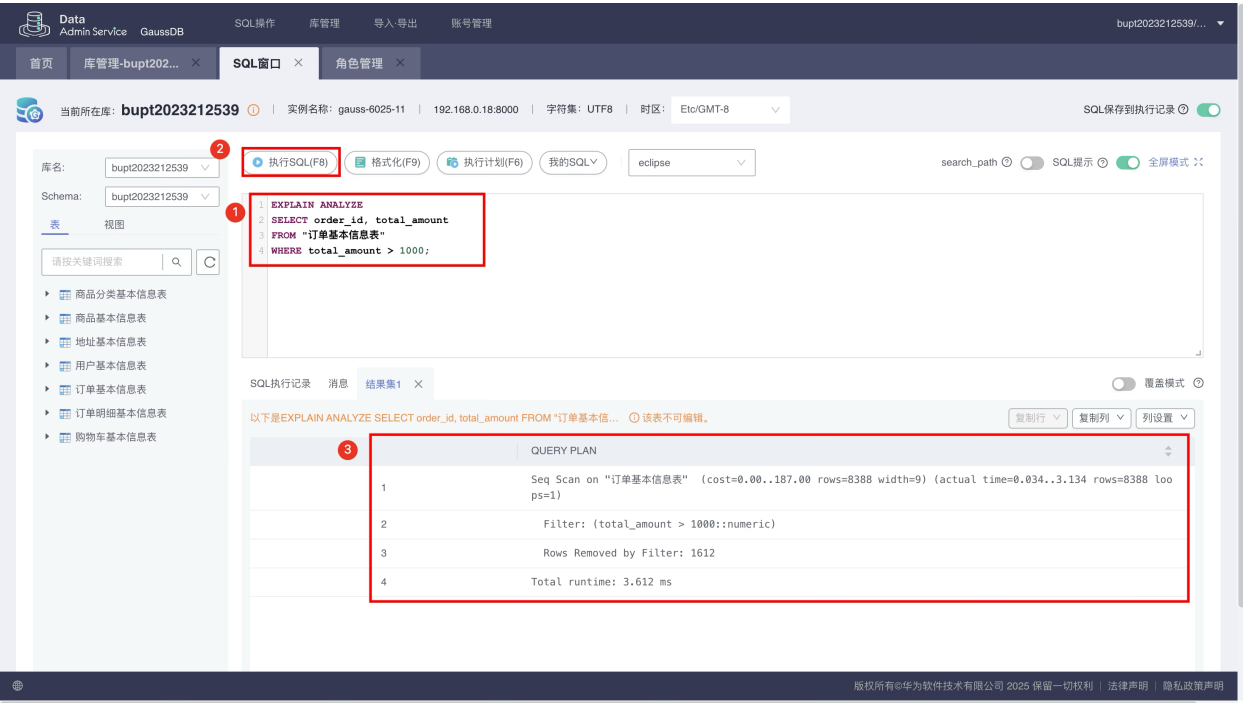


图 4-15 按照实验指导添加部分索引前查询时间

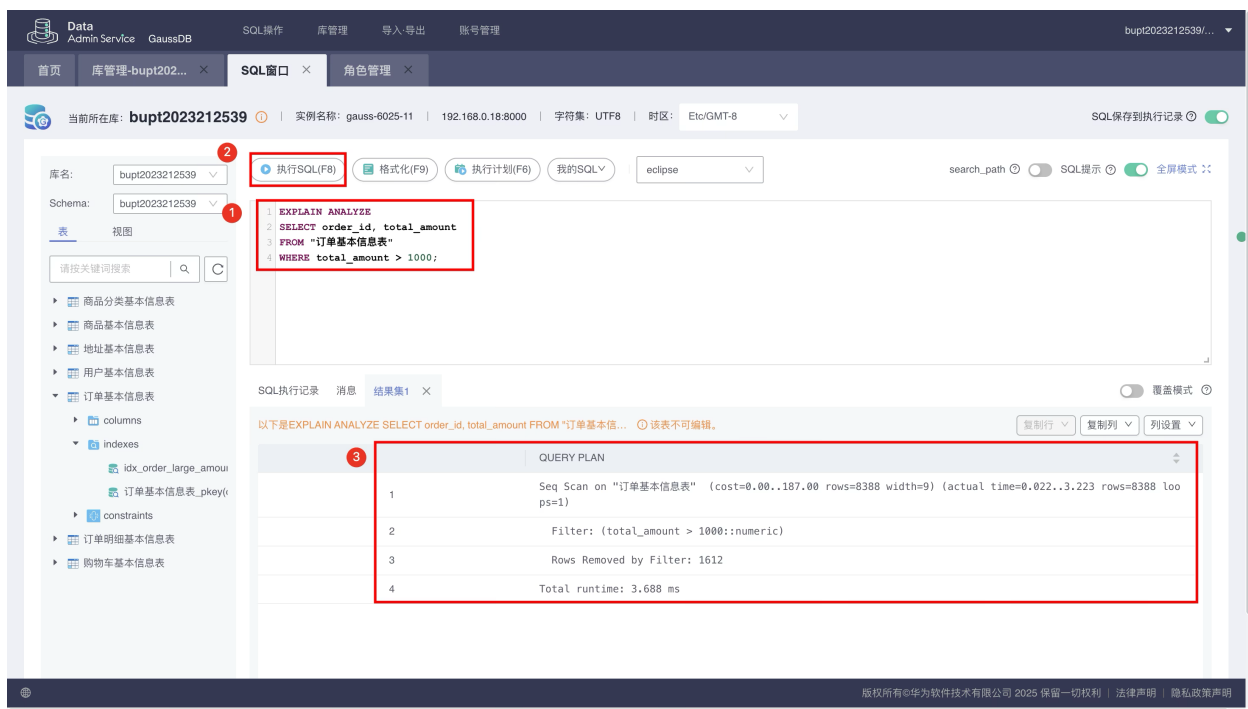


图 4-16 按照实验指导添加部分索引后查询时间

对于出现上述情况的原因分析如下：实际数据集导致索引的选择性极低，10000 行数据包含了 8388 行满足条件的行，导致优化器认为

- 使用索引需要先读取索引页，再根据索引中的指针去查找对应的主表数据行；
- 由于需要查找的行太多，进行大量的随机 I/O 来读取 80% 以上的数据，其开销反而高于直接进行一次顺序的、连续的全表扫描；

从两次查询的估计成本（`cost=0.00..187.00`）完全一致也可以验证，优化器在添加索引后，根据表的统计信息，依然认为使用 `Seq Scan` 的成本是最低的。

另外，由于创建的索引为 `ON "订单基本信息表" (order_id) WHERE total_amount > 1000;`。索引只存储满足条件的订单的 `order_id`。而我的查询是 `SELECT order_id, total_amount ...`。由于索引中不包含 `total_amount` 字段，数据库即使使用索引找到了匹配的 `order_id`，也必须回表去查找对应的 `total_amount`。

这种回表操作进一步增加了随机 I/O 的成本，使得优化器更倾向于直接进行 `Seq Scan`。

不过实际测试发现，回表操作并非优化器决策的决定性影响因素，本次实验场景下不影响选择结果。

4.3.3.2 步骤

结合上述分析，我更改业务场景为：需要经常查询总金额大于 50000 元的订单。使其匹配更少的数据行，从而提高选择性。

1. 在“订单基本信息表”中，对符合该条件的数据添加部分索引，对应的 SQL 语句见下文，执行结果如图 4-17 所示；

上文涉及的 SQL 语句如下：

```
1 CREATE INDEX idx_order_large_amount ON "订单基本信息表" (order_id) WHERE total_amount > 50000;
```

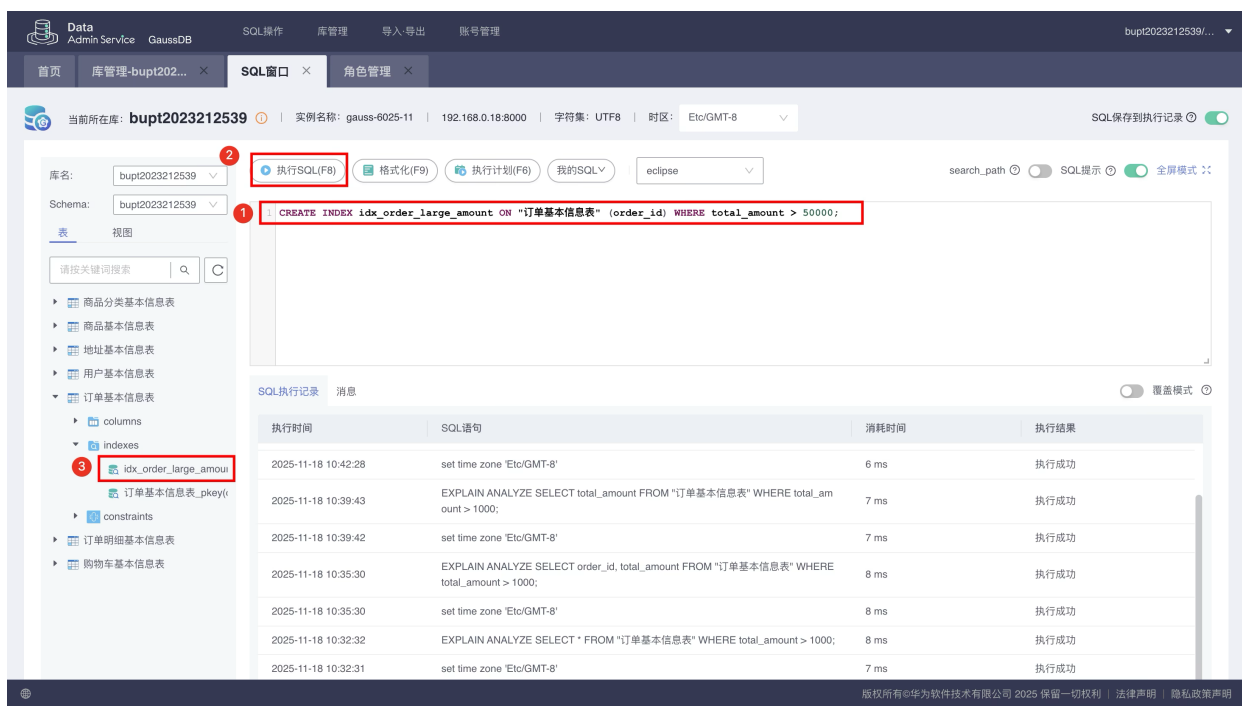


图 4-17 在“订单基本信息表”中，添加部分索引

2. 比较未建部分索引与创建部分索引后，查询效率的不同。这里，我使用 `EXPLAIN ANALYZE` 命令来执行测试查询，它会显示查询执行计划和实际运行时间（Total runtime），并选择了所有总金额大于 50000 元的订单作为查询对象。
 - (a) 测量无索引时的查询性能（基准测试），对应的 SQL 语句见下文，执行结果如图 4-18 所示；

- (b) 创建部分索引，见上文；
- (c) 测量有索引时的查询性能，在部分索引创建完成后，再次执行完全相同的查询，并记录新的运行时间进行比较，执行结果如图 4-19 所示；

上文涉及的 SQL 语句如下：

```
1 -- 找出所有总金额大于 50000 元的订单 ID 和总金额
2 EXPLAIN ANALYZE
3 SELECT order_id, total_amount
4 FROM "订单基本信息表"
5 WHERE total_amount > 50000;
```

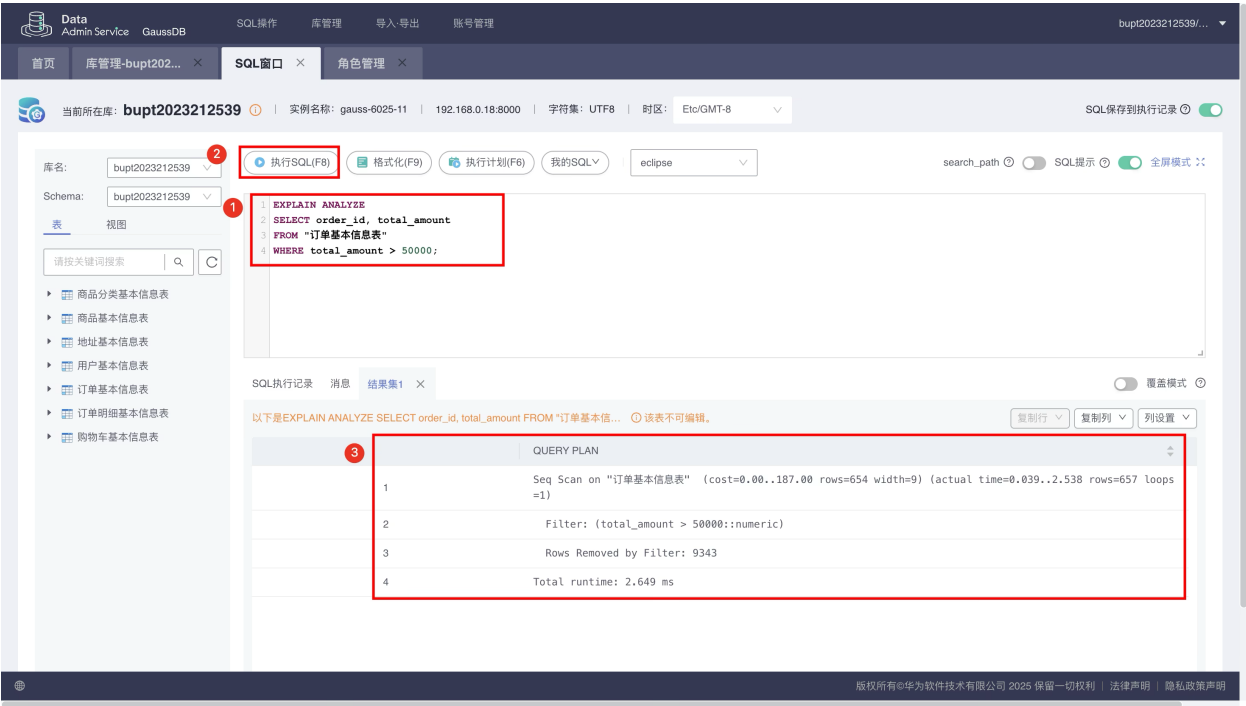


图 4-18 添加部分索引前查询时间

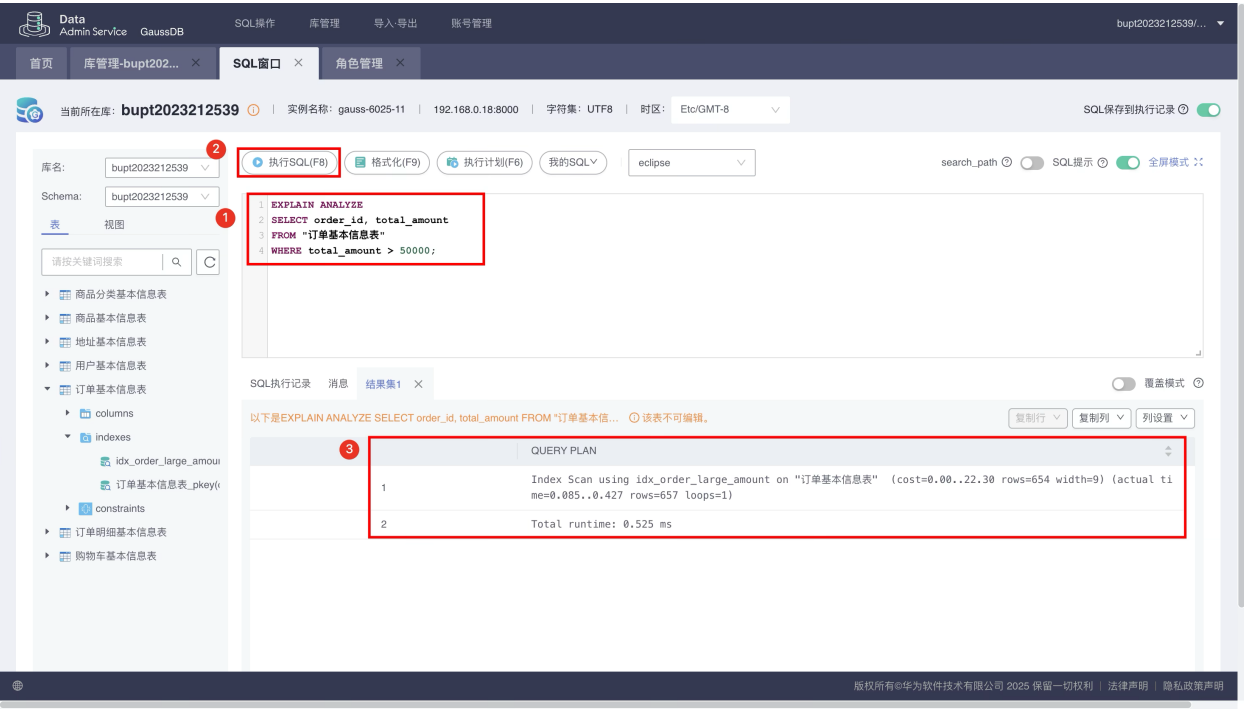


图 4-19 添加部分索引后查询时间

4.3.3.3 性能对比分析

整理上图所示的结果如表 4-3 所示：

表 4-3 添加部分索引前后查询效率对比

性能指标	未建索引 (Seq Scan)	已建部分索引 (Index Scan)	性能提升
查询方法	顺序扫描 (Seq Scan)	索引扫描 (Index Scan)	
查询成本	0.00..187.00	0.00..22.30	显著降低
实际执行时间	2.649 ms	0.525 ms	约 5.05 倍
过滤行数	9,343 行	直接通过索引定位	

这次的测试结果成功地证明了**高选择性**对于部分索引利用的重要性。通过将过滤条件从 `> 1000` 调整到 `> 50000`，使查询匹配的数据行比例大幅下降，优化器改变了决策，选择了更高效的索引扫描。

未利用索引时：

数据库进行了全表扫描，总行数约为 $657 + 9343 = 10,000$ 行。它扫描了 10,000 行并丢弃了 9,343 行。估计成本较高（187.00），总运行时间为 2.649 毫秒。

利用高选择性索引时：

数据库使用了新建的 `idx_order_large_amount` 部分索引。它直接在索引结构中定位到 `total_amount > 50000` 的订单 ID。估计成本大幅下降到 22.30，总运行时间降至 0.525 毫秒。

综上，当查询条件 `total_amount > 50000` 匹配的行数比例很低（约 6.57%）时，索引的选择性高，优化器判断执行**随机 I/O**（通过索引访问数据）的成本，低于执行顺序 I/O（全表扫描）的成本。通过正确利用高选择性部分索引，查询速度提升了约 **5.05 倍** ($2.649\text{ ms}/0.525\text{ ms} \approx 5.05$)。

这个实验结果证实了我之前的分析：数据库优化器只有在**索引能帮助它跳过大量不相关数据**时（即高选择性），才会选择使用索引，从而避免低效的全表扫描。

4.3.4 创建表达式索引

考虑以下业务场景：需要经常根据订单创建时间的年份进行查询。

- 1. 因此，在“订单基本信息表”中，对 `create_time` 建立**表达式索引**，对应的 SQL 语句见下文，执行结果如图 4-20 所示；

上文涉及的 SQL 语句如下：

```
1 CREATE INDEX idx_order_year ON "订单基本信息表" (EXTRACT(YEAR FROM create_time));
```

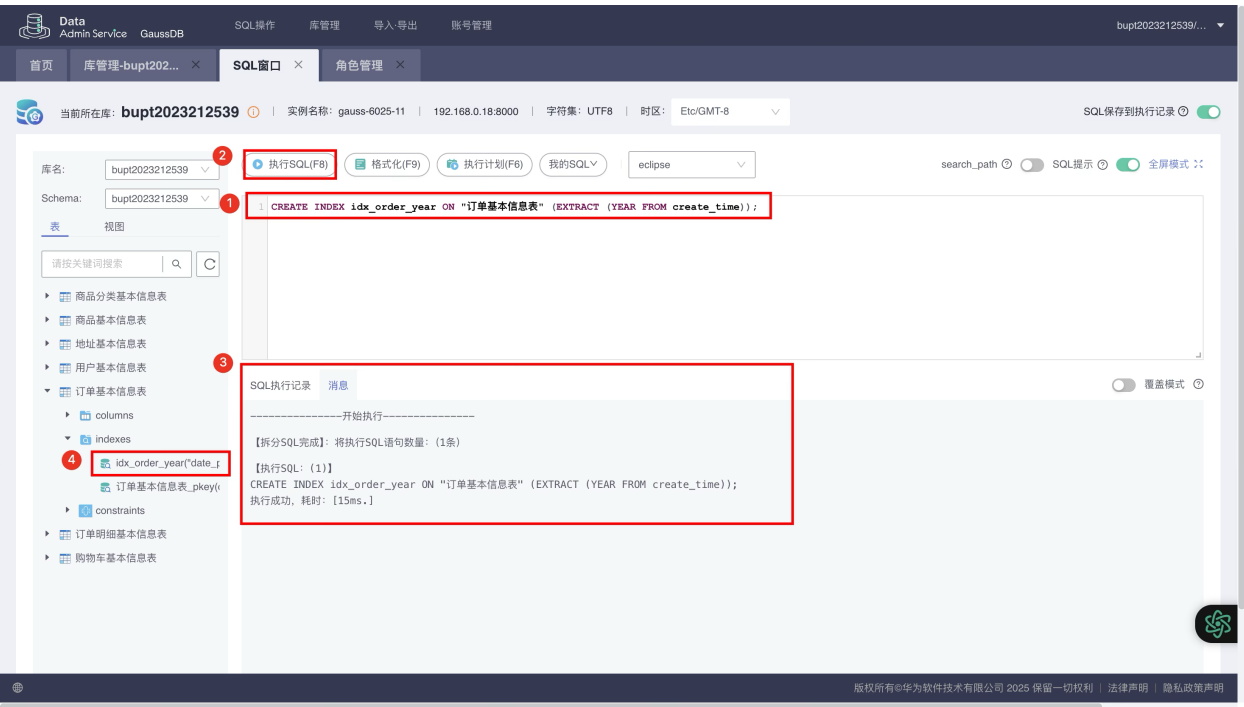


图 4-20 在“订单基本信息表”中，添加表达式索引

2. 比较未建表达式索引与创建表达式索引后，查询效率的不同。这里，我使用 `EXPLAIN ANALYZE` 命令来执行测试查询，它会显示查询执行计划和实际运行时间（Total runtime），并选择了所有创建时间的年份为 2023 年的订单作为查询对象。
- (a) 测量无索引时的查询性能（基准测试），对应的 SQL 语句见下文，执行结果如图 4-21 所示；
 - (b) 创建表达式索引，见上文；
 - (c) 测量有索引时的查询性能，在部分索引创建完成后，再次执行完全相同的查询，并记录新的运行时间进行比较，执行结果如图 4-22 所示；

上文涉及的 SQL 语句如下：

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM "订单基本信息表"
4 WHERE EXTRACT(YEAR FROM create_time) = 2023;
```

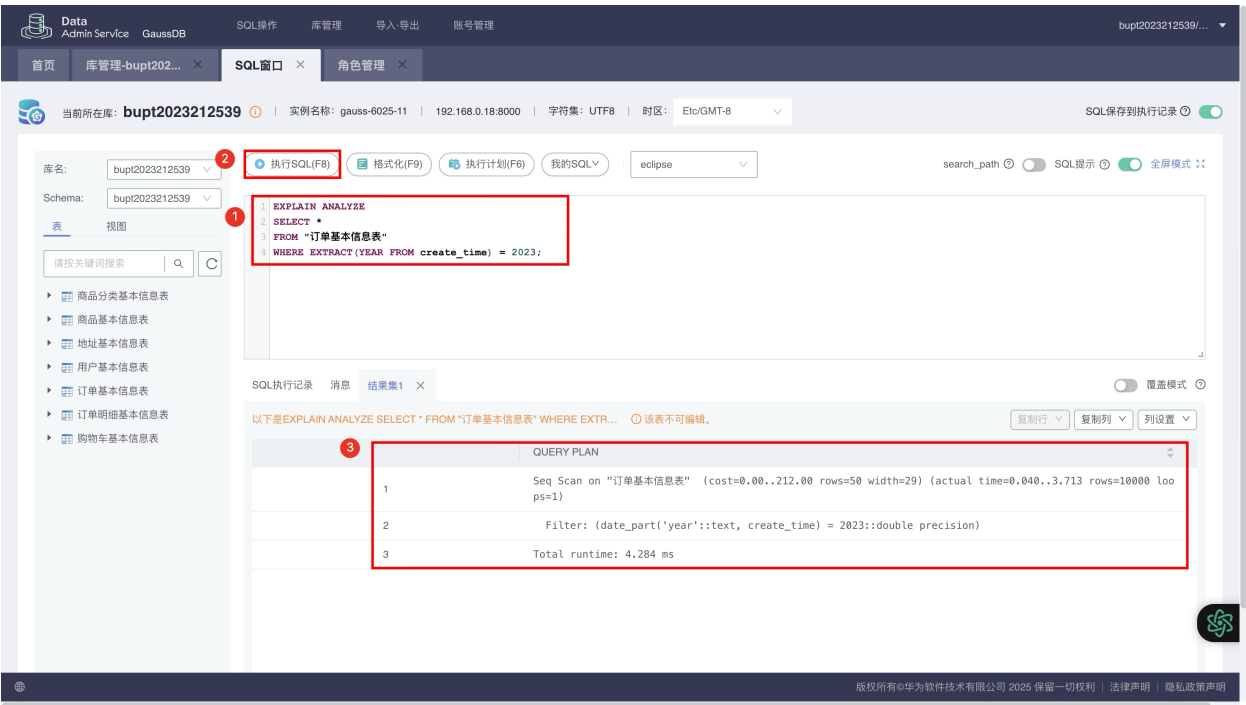


图 4-21 添加表达式索引前查询时间

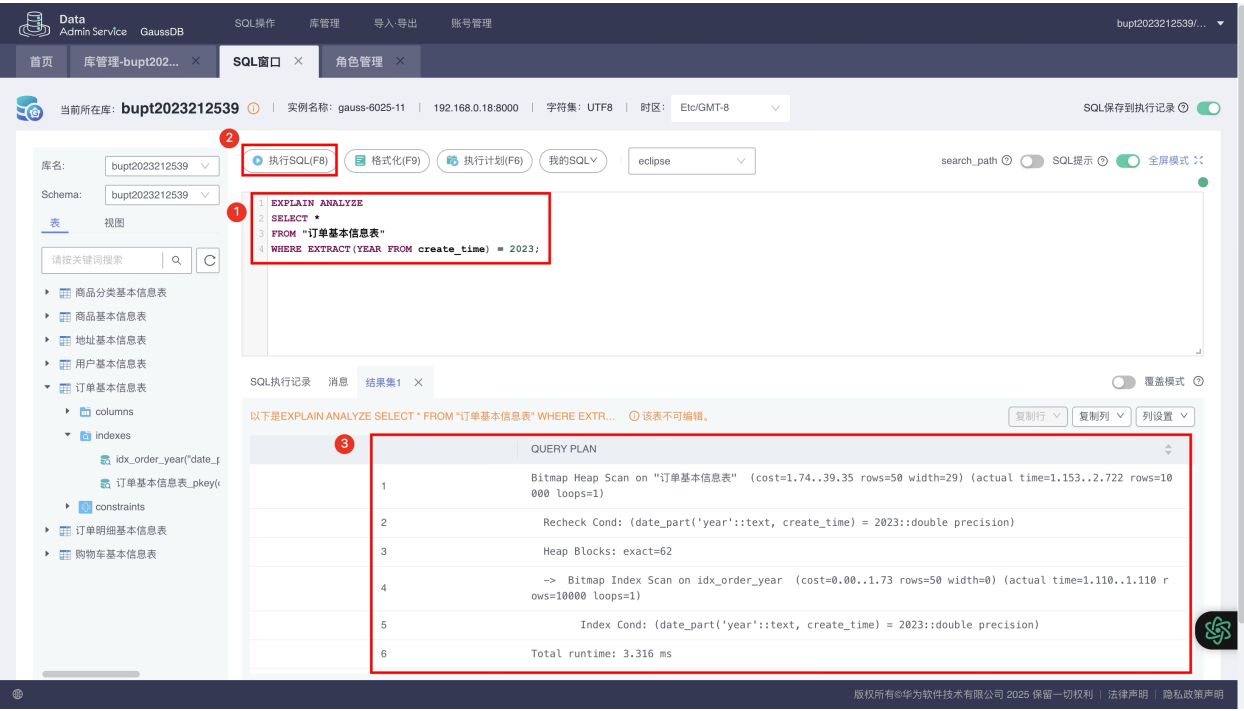


图 4-22 添加表达式索引后查询时间

4.3.4.1 性能对比分析

整理上图所示的结果如表 4-4 所示：

表 4-4 添加表达式索引前后查询效率对比

性能指标	未建索引 (Seq Scan)	已建表达式索引 (Bitmap Index Scan)	性能提升
查询方法	顺序扫描 (Seq Scan)	索引扫描 (Bitmap Heap Scan + Bitmap Index Scan)	
查询成本	0.00..212.00	1.74..39.35	显著降低
实际执行时间	4.284 ms	3.316 ms	约 1.29 倍

未建索引时：

数据库执行了全表顺序扫描 (Seq Scan)，遍历了所有 10,000 行数据。对于每一行，数据库都必须计算一次 `date_part('year'::text, create_time)`，然后将结果与 `2023` 进行比较。这种计算和比较是顺序进行的。因此估计成本较高 (212.00)，总运行时间为 4.284 毫秒。

已建表达式索引后

数据库使用了新建的表达式索引 `idx_order_year`。它使用了两步法：

1. **Bitmap Index Scan:** 首先，快速扫描索引结构，找出所有满足 `date_part('year'::text, create_time) = 2023` 条件的行所在的数据块（Heap Blocks）。
2. **Bitmap Heap Scan:** 然后，根据这些数据块的地址，高效地访问主表（Heap Blocks: exact=62），批量获取 10,000 行完整数据。

因此，估计成本大幅下降到 39.35，总运行时间为 3.316 毫秒。

综上，表达式索引成功地将函数计算的结果持久化，使得查询能够走索引路径。

4.4 创建和管理视图

基本概念：

- 当用户对数据库中的一张或者多张表的某些字段的组合感兴趣，而又不想每次键入这些查询时，用户就可以定义一个视图，以便解决这个问题。
- 视图与基本表不同，不是物理上实际存在的，是一个虚表。**数据库中仅存放视图的定义，而不存放视图对应的数据，这些数据仍存放在原来的基本表中。**若基本表中的数据发生变化，从视图中查询出的数据也随之改变。从这个意义上讲，视图就像一个窗口，透过它可以看到数据库中用户感兴趣的数据及变化。视图每次被引用时都会运行一次。

4.4.1 创建视图

1. 创建商品价格高于 5000 的 `high_price_products` 视图，对应的 SQL 语句见下文，执行结果如图 4-23 所示；
2. 创建所有电子产品的商品基本信息视图 `electronics_products`，包括商品编号、商品名、商品描述、商品价格、商品库存、商品类别编号，对应的 SQL 语句见下文，执行结果如图 4-24 所示；

上文涉及的 SQL 语句如下：

```
1  -- 创建 high_price_products 视图
2  CREATE VIEW high_price_products AS
3  SELECT * FROM "商品基本信息表" WHERE price > 5000;
```

```

4
5 -- 创建 electronics_products 视图
6 CREATE VIEW electronics_products AS
7 SELECT
8     p.product_id AS "商品编号",
9     p.name AS "商品名",
10    p.description AS "商品描述",
11    p.price AS "商品价格",
12    p.stock AS "商品库存",
13    p.category_id AS "商品类别编号"
14 FROM
15     "商品基本信息表" p
16 WHERE
17     p.category_id IN (
18         SELECT category_id
19         FROM "商品分类基本信息表"
20         WHERE
21             name = '电子产品'
22         UNION ALL
23         SELECT c.category_id
24         FROM "商品分类基本信息表" c
25         WHERE
26             c.parent_id = (SELECT category_id FROM "商品分类基本信息表" WHERE
27                             name = '电子产品')
28 );

```

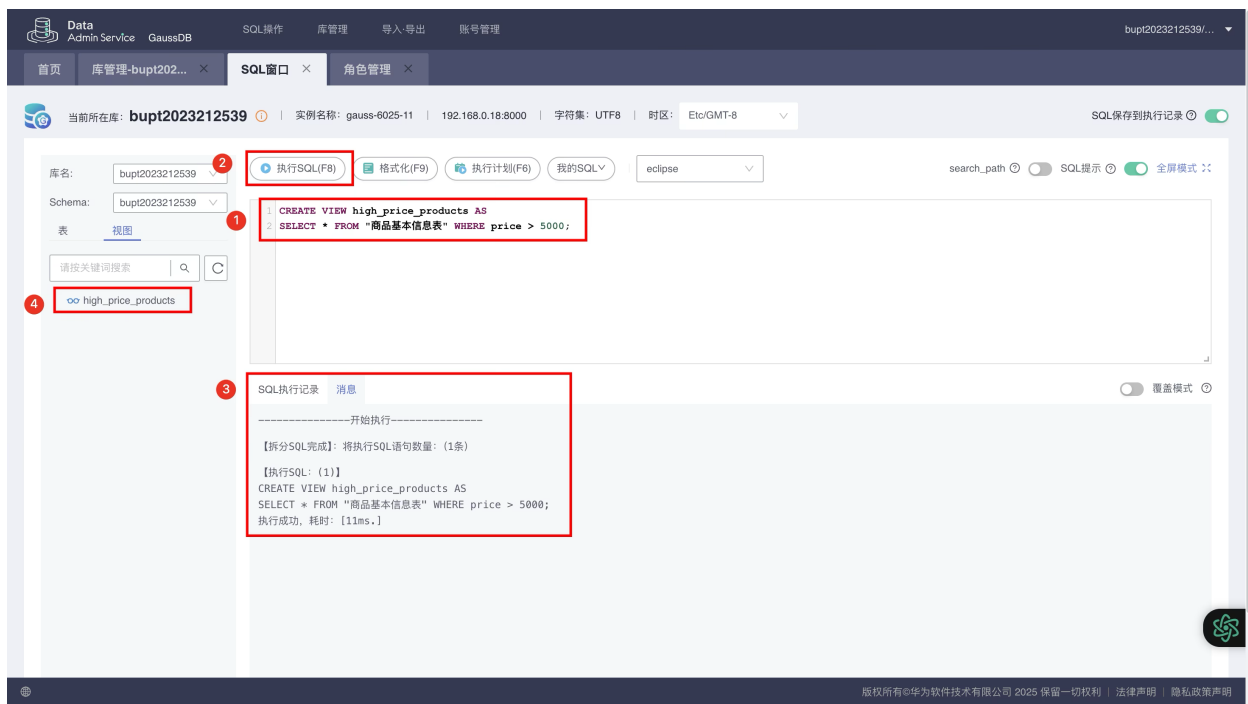


图 4-23 创建 high_price_products 视图

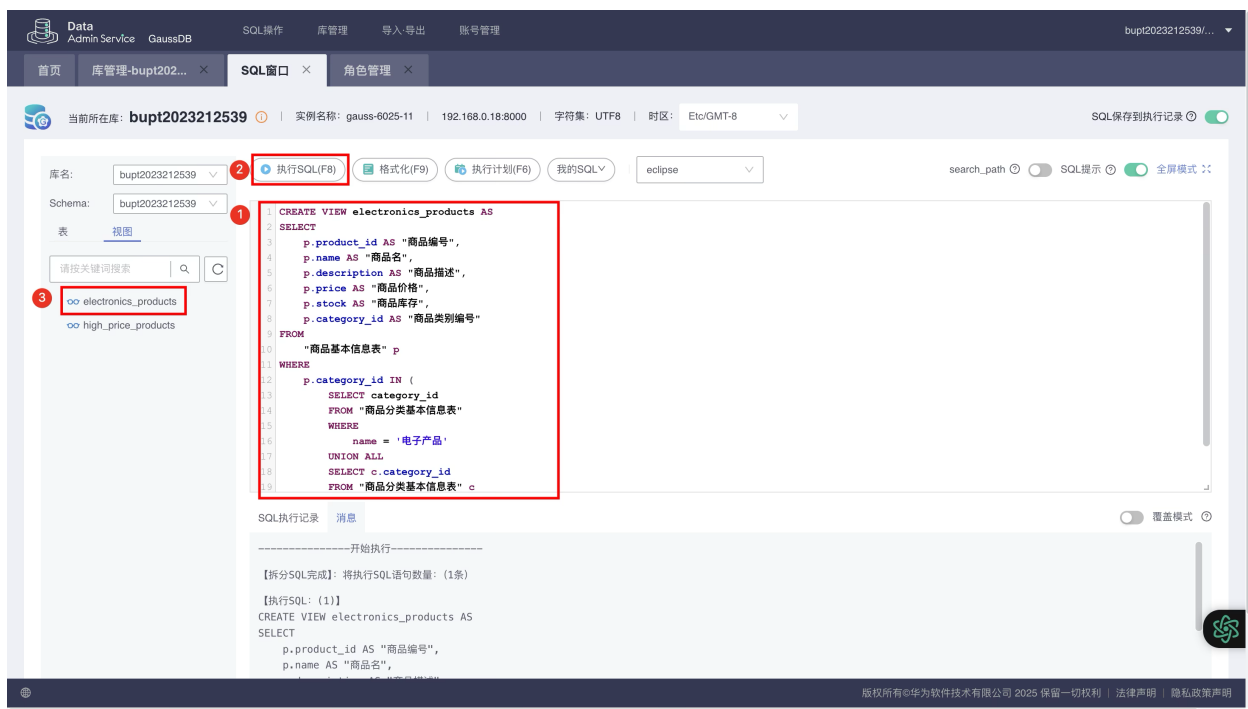


图 4-24 创建 electronics_products 视图

4.4.2 管理视图

1. 通过 `electronics_products` 视图查询价格在大于 300 小于 3000 的商品基本信息，按照商品编号和创建时间进行升序显示（截前五条记录）。

注：由于实验指导书中要求创建的视图并不包含创建时间，如果执行，会报如图 4-25 所示的错误“**ERROR: Column "create_time" does not exist.**”，我们在创建视图时并没有包含 `create_time` 列，因此虽然 `create_time` 存在于原表中，但当前用户不能通过该视图访问对应数据，体现了安全隔离。

2. 重新创建可以用于查询的视图，包含创建时间列，对应的 SQL 语句见下文，执行结果略；
3. 重新通过 `electronics_products` 视图查询价格在大于 300 小于 3000 的商品基本信息，按照商品编号和创建时间进行升序显示（截前五条记录），对应的 SQL 语句见下文，执行结果如图 4-26 所示，正确筛选出结果；

上文涉及的 SQL 语句如下：

```
1 -- 尝试查询不在 electronics_products 视图中的列
2 SELECT
3     "商品编号",
4     "商品名",
5     "商品描述",
6     "商品价格",
```

```
7      "商品库存",
8      "商品类别编号",
9      create_time
10 FROM
11     electronics_products
12 WHERE
13     "商品价格" > 300 AND "商品价格" < 3000
14 ORDER BY
15     "商品编号" ASC,
16     create_time ASC
17 LIMIT 5;
18
19 -- 重新创建能满足查询条件的视图
20 DROP VIEW IF EXISTS electronics_products;
21 CREATE VIEW electronics_products AS
22 SELECT
23     p.product_id AS "商品编号",
24     p.name AS "商品名",
25     p.description AS "商品描述",
26     p.price AS "商品价格",
27     p.stock AS "商品库存",
28     p.category_id AS "商品类别编号",
29     p.create_time AS "创建时间"
30 FROM
31     "商品基本信息表" p
32 WHERE
33     p.category_id IN (
34         SELECT category_id
35         FROM "商品分类基本信息表"
36         WHERE
37             name = '电子产品'
38         UNION ALL
39         SELECT c.category_id
40         FROM "商品分类基本信息表" c
41         WHERE
42             c.parent_id = (SELECT category_id FROM "商品分类基本信息表" WHERE
name = '电子产品')
43     );
44
45 -- 重新执行查询
46 SELECT
47     "商品编号",
48     "商品名",
49     "商品描述",
50     "商品价格",
51     "商品库存",
52     "商品类别编号",
53     "创建时间"
54 FROM
55     electronics_products
56 WHERE
57     "商品价格" > 300 AND "商品价格" < 3000
58 ORDER BY
59     "商品编号" ASC,
60     "创建时间" ASC
```

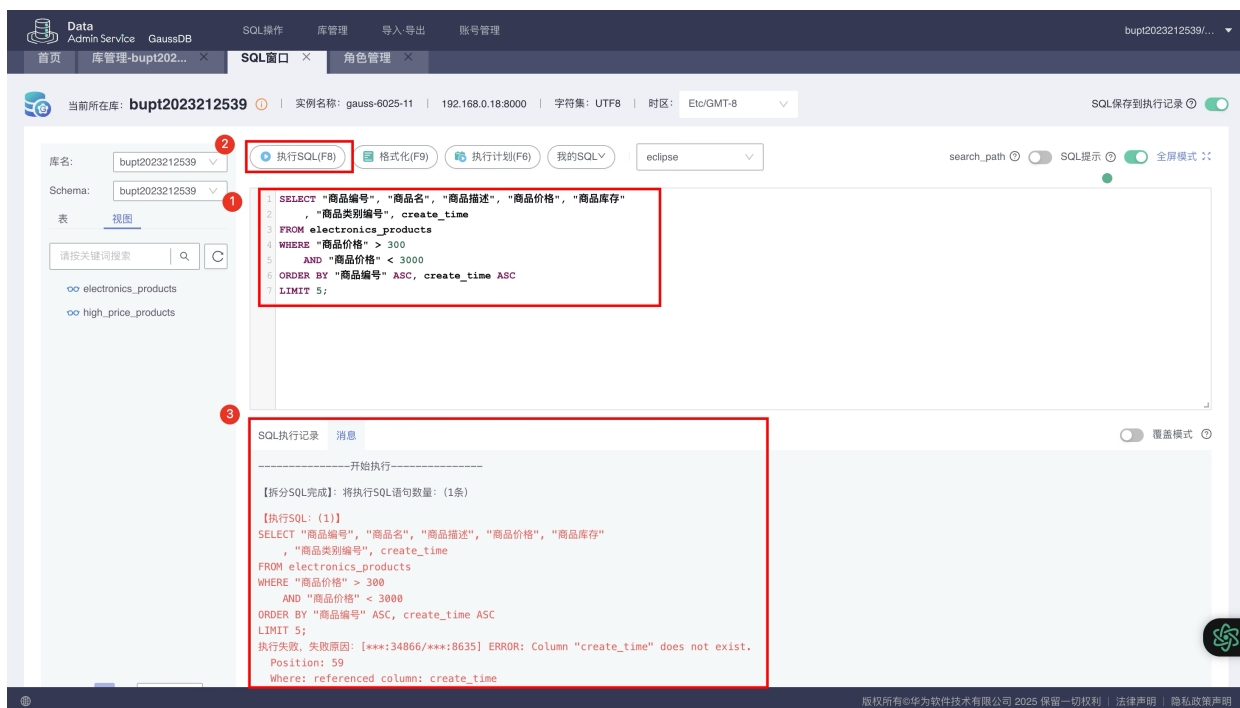
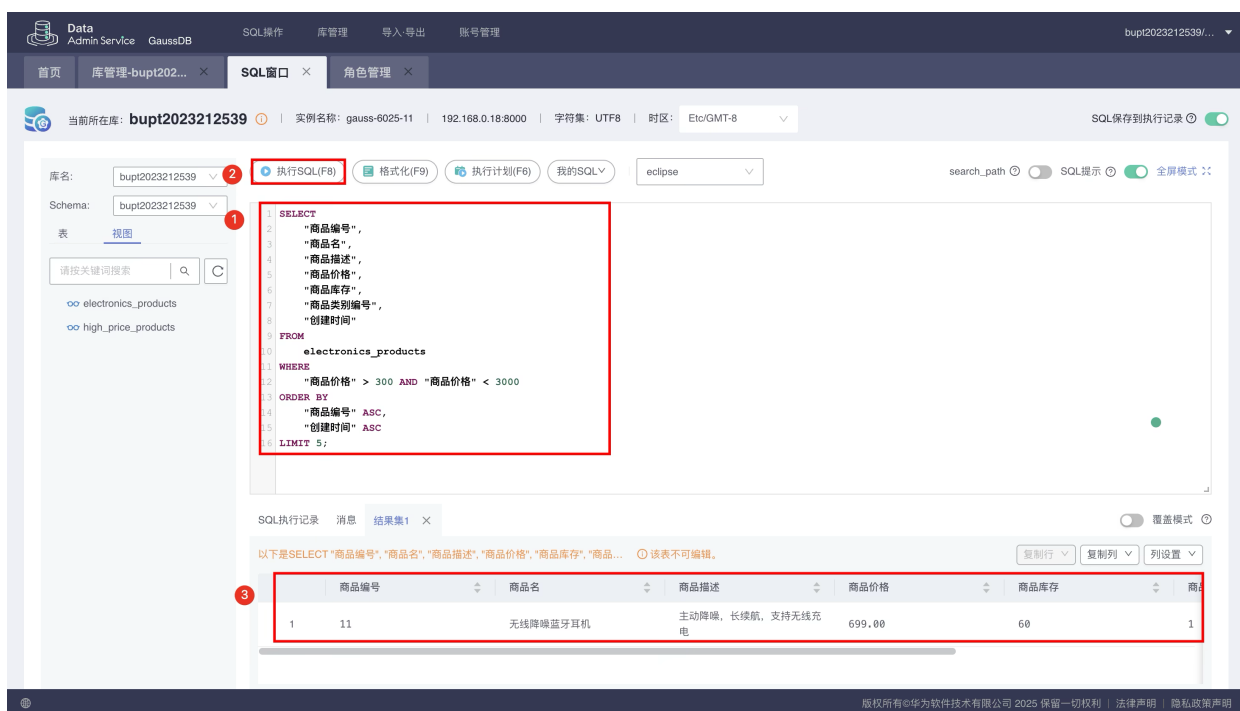

图 4-25 尝试查询不在 `electronics_products` 视图中的列

图 4-26 正确创建后重新执行查询

4. 查询 `high_price_products` 视图，对应的 SQL 语句见下文，执行结果如图 4-27 所示，正确筛选出结果；
5. 查看视图定义，对应的 SQL 语句见下文，执行结果如图 4-28 所示；

注：这里实验指导书给出 SQL 语句（`SHOW CREATE VIEW`）有误，这是 MySQL 的操作，PostgreSQL 并不支持这一点。

6. 删除视图，对应的 SQL 语句见下文，执行结果如图 4-29 所示；

```
1 SELECT * FROM high_price_products;
2
3 SELECT pg_get_viewdef('high_price_products', true);
4
5 DROP VIEW high_price_products;
```

库名: bupt2023212539

Schema: bupt2023212539

表

视图

请按关键词搜索

商品分类基本信息表

商品基本信息表

地址基本信息表

用户基本信息表

订单基本信息表

订单明细基本信息表

购物车基本信息表

执行SQL(F8)

格式化(F9)

执行计划(F6)

我的SQL

eclipse

search_path

SQL提示

全屏模式

SELECT * FROM high_price_products;

SQL执行记录 消息 结果集1

以下是SELECT * FROM high_price_products的执行结果集

	product_id	name	description	price	stock	cat
1	8	苹果iPad Pro	12.9英寸全面屏, M1芯片, 高性能平板电脑	7999.00	50	1
2	9	苹果iPhone 14 Pro	6.1英寸超视网膜XDR显示屏, A16仿生芯片	8999.00	30	1
3	10	高性能轻薄笔记本电脑	16英寸, Intel i7处理器, 16GB内存	8999.00	25	1
4	12	4K超高清智能电视	55英寸, 支持HDR10, 内置智能系统	5999.00	20	2
5	13	变频中央空调挂机	节能静音, 风冷无霜, 大容量设计	6000.00	15	3

当前第 1 页 50 条/页 跳转到 1 共 5 条 获取总行数

图 4-27 正确创建后重新执行查询

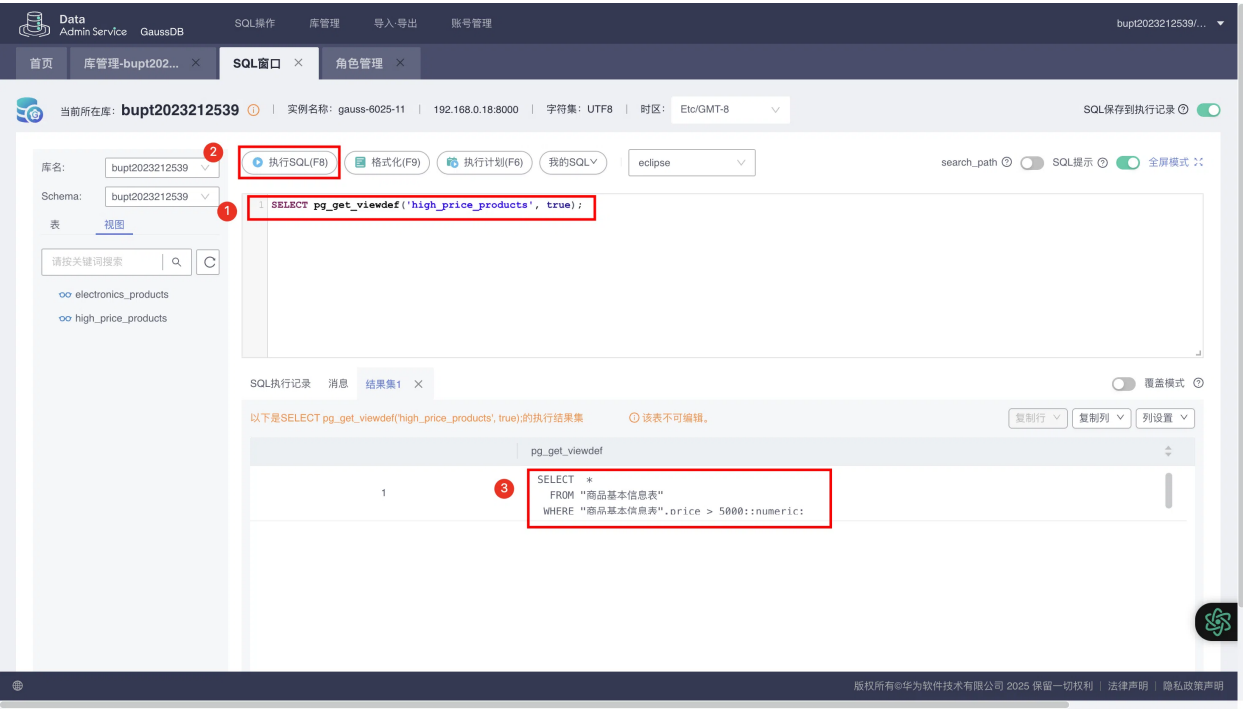


图 4-28 查看 high_price_products 视图定义

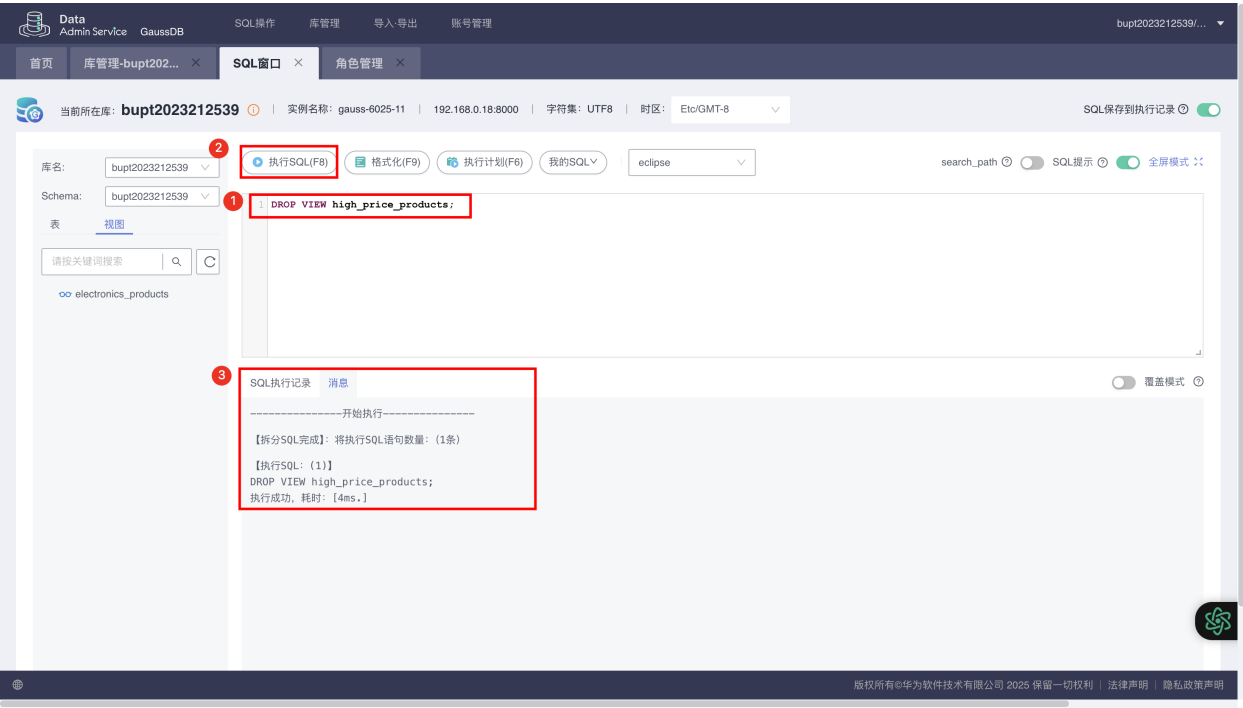


图 4-29 删除 high_price_products 视图

5 实验总结与心得体会

本次实验中，我深入实践了 GaussDB 数据库中的索引管理和视图管理，初步掌握了 SQL 相关的索引和视图语法，并且了解了几种不同类型的索引，也让我对数据库中的实际应用有了更多了解。通过一系列的创建、管理、性能对比和故障排除操作，我不仅学习了语法，更深化了对数据库查询优化器工作原理的理解。

实验中遇到的主要问题为索引选择性与优化器决策。在创建部分索引时，我发现按照实验指导创建的 `WHERE total_amount > 1000` 的索引没有生效，查询依然执行了成本较高的顺序扫描。通过分析 `EXPLAIN ANALYZE` 结果，我确认了这是因为匹配条件的行数过多（约 83.88%），导致索引的选择性极低。优化器认为进行大量随机 I/O 的成本高于顺序扫描全表的成本。随后，通过将条件调整为 `WHERE total_amount > 50000`，使选择性提高，索引成功生效，查询性能提升。通过实验，深刻地证明了索引并非越多越好，其效用高度依赖于查询条件与数据分布的匹配程度。

此外，通过创建并管理视图，我更加理解了视图的作用，它不仅能简化复杂的查询，更能作为一种安全隔离层或封装机制。用户只能通过视图的定义来访问特定的列和数据，无法看到基础表的所有结构，使得数据库在面向用户时更加方便便捷。

一些对实验的建议：

1. 增强索引选择性设计的指导：建议在实验指导中增加更明确的说明，强调测试索引性能时，应选择具有高选择性的查询条件，以避免学生对索引的性能提升产生误解。
2. 统一 SQL 命令的准确性：建议在实验指导中修正所有不适用于 PostgreSQL/GaussDB 的命令（如 `SHOW CREATE VIEW`），统一使用如 `pg_get_viewdef` 等正确命令，以确保实验环境与操作的一致性。